HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY



GRADUATION THESIS

Development of a Vietnamese Legal Question Answering System using Agentic Retrieval-Augmented Generation and Fine-Tuned Language Models

Student: *Bùi Văn Huy - 20225497*

Email: *huy.bv225497@sis.hust.edu.vn*

Program: *Data Science and AI*

Class: *762001*

Lecturer: *Assoc. Prof. Nguyễn Quốc Tuấn*

Hanoi, Jun 2026

ACKNOWDGEMENT

First and foremost, I would like to express my deepest and most sincere gratitude to my supervisor, Lecturer Nguyen Quoc Tuan, who has directly guided and dedicatedly instructed me throughout the implementation of this graduation project. Without his orientation and invaluable feedback, I would not have been able to successfully complete this thesis.

I also extend my sincere thanks to the Board of Management of Hanoi University of Science and Technology, as well as all the professors and lecturers at the School of Information and Communication Technology (SOICT), for imparting invaluable specialized knowledge to me during my years of study at the university.

Furthermore, I would like to express my profound gratitude to my family and friends—those who have always provided the best conditions, moral support, and accompanied me throughout my academic journey and the realization of this project.

The completion of this graduation project marks a memorable milestone in my academic journey. However, due to limitations in time and practical experience, this thesis inevitably contains shortcomings. I highly welcome and look forward to receiving constructive feedback and suggestions from the professors to further improve this research.

I wish all the professors and lecturers abundant health and continued success in their noble educational careers.

I sincerely thank you!

ABSTRACT

Accessing and comprehending Vietnamese legal information is a significant challenge due to the complexity, specialized terminology, and cross-referencing nature of legal documents. While Large Language Models (LLMs) and traditional Retrieval-Augmented Generation (RAG) systems have been employed to automate legal question-answering, they often suffer from hallucinations and struggle with multi-step reasoning due to their static, linear retrieval mechanisms. To address these limitations, this thesis proposes an advanced approach utilizing Agentic RAG combined with fine-tuned sub-models. This approach was selected because autonomous agents can dynamically plan, self-evaluate, and refine search strategies, thereby significantly improving context retrieval and response accuracy in complex legal scenarios while minimizing LLM token usage. The proposed system is orchestrated by a central Orchestrator managing specialized agents: a Planner Agent for unified query routing, rewriting, and metadata entity extraction; a Grader Agent for evaluating retrieval quality; and a Hallucination Checker to ensure responses are strictly grounded in the retrieved context. To optimize the pipeline, three sub-models were fine-tuned specifically on Vietnamese legal datasets: a Sentence Transformer for semantic embedding, a Cross-Encoder for document re-ranking, and a BERT model for precise information extraction. The main contribution of this research is the development of a robust, cost-effective, and highly accurate legal question-answering system. Experimental results on a dataset of 722 real-world legal questions demonstrated significant improvements, achieving an average Cosine Similarity of 0.6809, a ROUGE-L score of 0.3480, and a Token-F1 score of 0.1744, with 100% routing accuracy for legal intents. Finally, the system was successfully deployed into a practical web application using FastAPI and Docker, featuring real-time reasoning visualization via Server-Sent Events (SSE) and conversational memory management.

Student

Huy

Bui Van Huy

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION **1**

1.1 Problem Statement 1

1.2 Background and Problems of Research 1

1.3 Research Objectives and Conceptual Framework 2

1.4 Contributions 2

CHAPTER 2. LITERATURE REVIEW **4**

2.1 Scope of Research 4

2.2 Related Work 4

2.3 Theoretical Background: Transformer and Large Language Models 7

2.4 Theoretical Background: Model Fine-Tuning Techniques 10

2.5 Theoretical Background: Retrieval-Augmented Generation Architectures 12

2.6 Theoretical Background: Evaluation Metrics 12

CHAPTER III. METHODOLOGY **14**

3.1 Overview 14

3.1.1 Offline knowledge preparation 14

3.1.2 Online inference workflow 15

3.2 Legal Answer Extraction Model 15

3.2.1 Problem definition 15

3.2.2 Dataset construction 16

3.2.3 Preprocessing and label generation 16

3.2.4 Model formulation 17

3.2.5 Fine-tuning strategies 18

3.2.6 Training configuration 19

3.3 Embedding Model for Semantic Retrieval 19

3.3.1 Motivation 19
3.3.2 Dataset construction 20
3.3.3 Vietnamese lexical normalization 21
3.3.4 Data organization for retrieval-oriented evaluation 21
3.3.5 Model architecture 21
3.3.6 Training objective 22
3.3.7 Training configuration 23
3.4 Reranking Model for Retrieved Legal Passages 23
3.4.1 Motivation 23
3.4.2 Dataset construction 24
3.4.3 Preprocessing 24
3.4.4 Model architecture and objective 24
3.4.5 Training configuration 26
3.5 Legal Knowledge Base Construction 26
3.5.1 Document collection 26
3.5.2 Organization and metadata preparation 27
3.5.3 Content cleaning and structural parsing 27
3.5.4 Tokenization, semantic chunking, and vector storage . . . 29
3.6 Agentic RAG Orchestration 30
3.6.1 System design 30
3.6.2 Planner Agent 32
3.6.3 Execution branches 32
3.6.4 Internal legal RAG pipeline 32
3.6.5 Reflective grading and iterative retrieval 33
3.6.6 Generation and hallucination control 34
3.6.7 Web-search fallback 34
3.7 Runtime Implementation 35
3.7.1 Conversation memory 35

3.7.2 Persistence layer 35
3.7.3 Backend, streaming, and deployment 35

CHAPTER IV. Evaluation and Results **37**
4.1 Evaluation Parameters 37
4.1.1 Metrics for extractive answer extraction 37
4.1.2 Metrics for embedding retrieval 37
4.1.3 Metrics for reranking 38
4.1.4 Metrics for Planner Agent and end-to-end evaluation . . . 38
4.1.5 Simulation Method 39
4.1.6 Datasets and data preparation 39
4.1.7 Training configuration and component setup 41
4.1.8 Integrated system evaluation protocol 42
4.2 Experimental Results on Legal Information Extraction 43
4.3 Experimental Results on Embedding-Based Retrieval 45
4.4 Experimental Results on Reranking 47
4.5 Experimental Results on the Planner Agent and the Integrated Agentic RAG System 48
4.5.1 Planner Agent evaluation 48
4.5.2 End-to-end Agentic RAG evaluation 50

Conclusion and Future Work **52**
5.1 Conclusion 52
5.2 Future Work 53

REFERENCE **54**

CHAPTER 1. INTRODUCTION

This chapter introduces the fundamental context and motivation behind the research on an intelligent legal question-answering system. It begins by defining the problem statement, highlighting the complexities of accessing Vietnamese legal information and the limitations of utilizing pure Large Language Models (LLMs) in this domain. Subsequently, the chapter discusses the theoretical background and identifies the drawbacks of existing Retrieval-Augmented Generation (RAG) frameworks. Based on these identified issues, the research objectives and the conceptual framework utilizing the Agentic RAG architecture are established. Finally, the chapter outlines the main contributions of this thesis and provides a structural overview of the remaining chapters.

1.1 Problem Statement

In modern society, the demand for rapid and accurate legal information retrieval is increasingly critical. Previously, accessing legal information relied heavily on consulting experts or manually searching through official sources. This traditional approach is highly time-consuming and requires profound domain knowledge. Specifically, Vietnamese legal documents possess complex structures, specialized terminology, and numerous exceptions, making it exceedingly difficult for ordinary users to identify applicable regulations for their specific situations.

The rapid development of Artificial Intelligence (AI) and Natural Language Processing (NLP) has opened new opportunities for automated legal information retrieval. Large Language Models (LLMs) have been widely adopted to address this demand; however, they frequently suffer from "hallucinations" when processing complex legal issues, generating plausible but factually incorrect responses. Since legal consulting requires absolute precision, relying solely on standalone LLMs poses significant risks. Therefore, developing a robust system capable of not only understanding deep legal contexts but also retrieving and applying accurate legal clauses is a pressing necessity to modernize legal services and make legal information more accessible to the public.

1.2 Background and Problems of Research

To mitigate the hallucination problem of LLMs, Retrieval-Augmented Generation (RAG) was introduced as a significant technological advancement. Traditional RAG systems combine an LLM with an external vector database, allowing the

model to retrieve relevant documents before generating a response. This mechanism provides the LLM with factual context, thereby improving the accuracy and timeliness of the answers.

Despite these improvements, traditional RAG systems still exhibit critical limitations. They are inherently linear and static in design, meaning they execute a fixed pipeline of retrieval followed by generation. This static nature renders them incapable of performing multi-step reasoning or adapting dynamically to highly complex legal contexts. Furthermore, the quality of the final response is heavily dependent on the initial retrieval phase. If the embedding or retrieval mechanism is inefficient, the system may retrieve irrelevant documents, leading the LLM to generate inaccurate or out-of-context answers. Consequently, a more dynamic and self-correcting approach is required to handle the rigorous demands of legal question-answering.

1.3 Research Objectives and Conceptual Framework

The primary objective of this research is to develop an intelligent legal question-answering system utilizing the advanced Agentic RAG architecture. This approach aims to overcome the static limitations of traditional RAG by introducing autonomous agents capable of dynamic planning, self-evaluation, and iterative retrieval.

To achieve this goal, the conceptual framework focuses on two main components. First, the project involves fine-tuning three specialized sub-models to optimize the retrieval pipeline: an Information Extraction model (BERT), a Re-ranking model (Cross-Encoder), and a Semantic Embedding model (Sentence Transformer). Second, the framework integrates these optimized sub-models into an Agentic RAG system orchestrated by a central module. This Orchestrator coordinates specialized agents, including a Planner Agent for query routing, a Grader Agent for assessing retrieval quality, and a Hallucination Checker for verifying response accuracy. This comprehensive approach ensures that the system can deeply understand legal contexts, retrieve precise information, and deliver reliable legal solutions.

1.4 Contributions

This thesis provides a comprehensive solution for automated legal consulting in Vietnam. The main contributions of this project are summarized as follows:

1. The project proposes and implements an Agentic RAG architecture tailored for the Vietnamese legal domain, significantly improving multi-step reasoning and dynamic context retrieval compared to traditional linear RAG systems.

2. The thesis successfully fine-tunes three critical sub-models (Embedding, Reranking, and Information Extraction) specifically on Vietnamese legal datasets, thereby enhancing retrieval accuracy and minimizing computational costs during the generation phase.

3. The research delivers a fully functional, web-based legal question-answering platform deployed via FastAPI and Docker, featuring real-time reasoning visualization and conversational memory management to support practical, real-world usage.

In summary, this chapter has outlined the critical need for an accurate and accessible legal question-answering system in Vietnam. It identified the core limitations of pure LLMs and traditional RAG frameworks, particularly their inability to handle complex, multi-step legal reasoning. By defining clear research objectives focused on implementing an Agentic RAG architecture and fine-tuning specialized sub-models, the foundation for this thesis has been established.

CHAPTER 2. LITERATURE REVIEW

This chapter establishes the theoretical foundation and provides a comprehensive review of the literature relevant to the development of an intelligent legal question-answering system. It begins by defining the scope of the research. Subsequently, it analyzes existing methodologies and related works in the field of automated question-answering, highlighting their strengths and limitations to formulate the motivation for this thesis. Finally, the chapter delves into the core theoretical concepts that underpin the proposed system, including the Transformer architecture, Large Language Models (LLMs), fine-tuning techniques, and the evolution of Retrieval-Augmented Generation (RAG) frameworks.

2.1 Scope of Research

This research focuses on the application of Natural Language Processing (NLP) techniques specifically tailored for the Vietnamese language within the legal domain. The scope is strictly bounded to processing and retrieving information from Vietnamese legal documents, including laws, decrees, and practical legal scenarios. The datasets utilized for model training and evaluation are curated from official and reliable sources, such as the National Database of Legal Documents and online legal consulting platforms. Technologically, the research is confined to the implementation of the Agentic RAG architecture and the fine-tuning of specific open-source models—namely BERT for information extraction, Sentence Transformers for semantic embedding, and Cross-Encoders for document re-ranking. The system aims to assist users in retrieving legal references and answering situational queries, rather than replacing professional legal counsel.

2.2 Related Work

The development of automated question-answering (QA) systems has progressed through several distinct paradigms, each presenting unique advantages and inherent limitations. Early approaches heavily relied on Scripted Chatbots, which operate based on predefined rules and decision trees. While these systems are easy to deploy and suitable for highly structured, repetitive tasks, their rigid nature renders them ineffective when handling complex, nuanced, or out-of-scope legal queries.

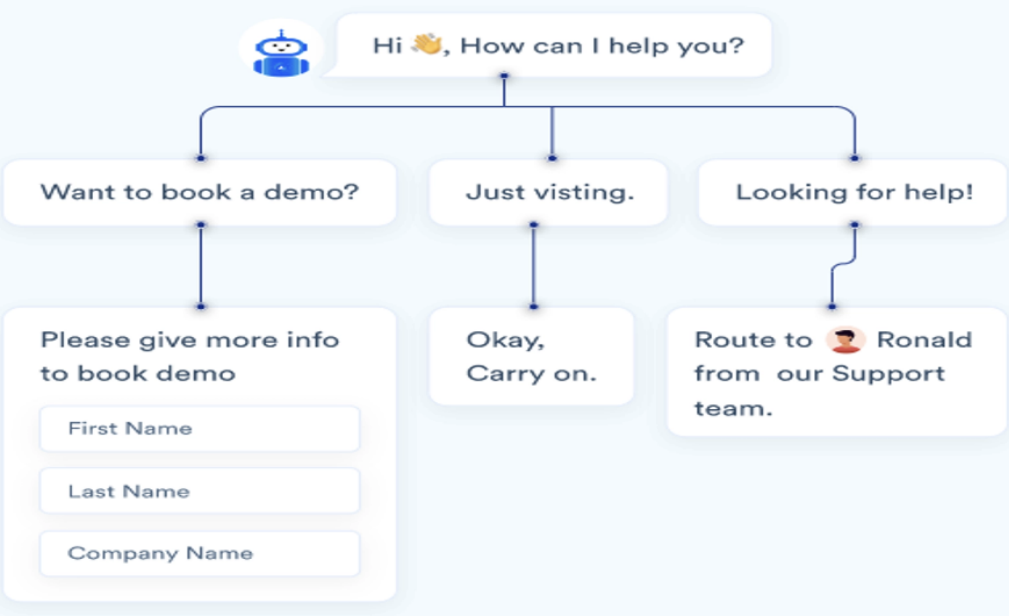


Figure 1 Scripted Chatbot workflow

To overcome the limitations of rule-based systems, researchers introduced QA systems based on Knowledge Graphs (KGs). These systems represent information as entities and relationships, allowing for logical deduction and high precision in specialized domains. However, the construction, maintenance, and expansion of KGs require massive manual effort and domain expertise. In a rapidly changing field like law, keeping a Knowledge Graph continuously updated is computationally and financially prohibitive.

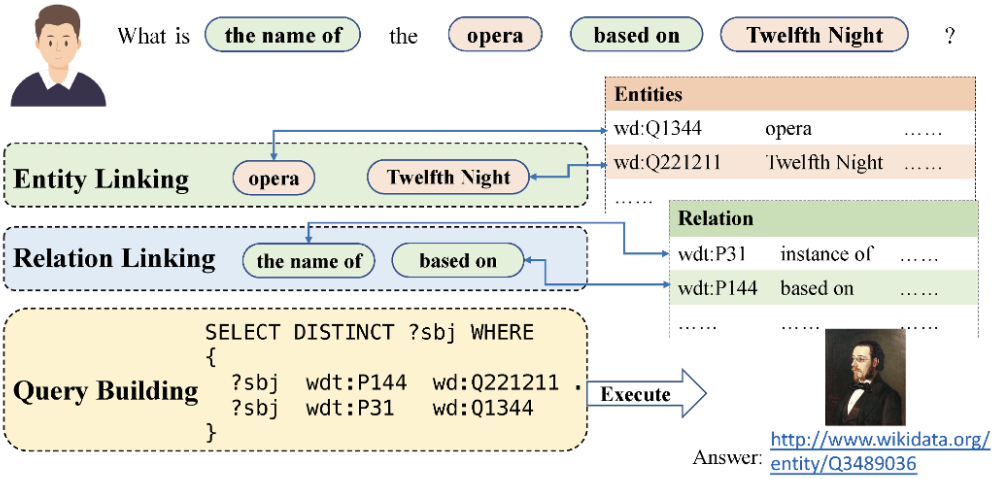


Figure 2 Knowledge graph workflow

Recently, the integration of Large Language Models (LLMs) with vector databases, known as Retrieval-Augmented Generation (RAG), has become the standard for

knowledge-intensive NLP tasks. Standard RAG systems retrieve relevant external documents to ground the LLM's responses, thereby reducing hallucinations. Despite this, traditional RAG architectures operate linearly and statically, lacking the capability to perform multi-step reasoning or self-correction.

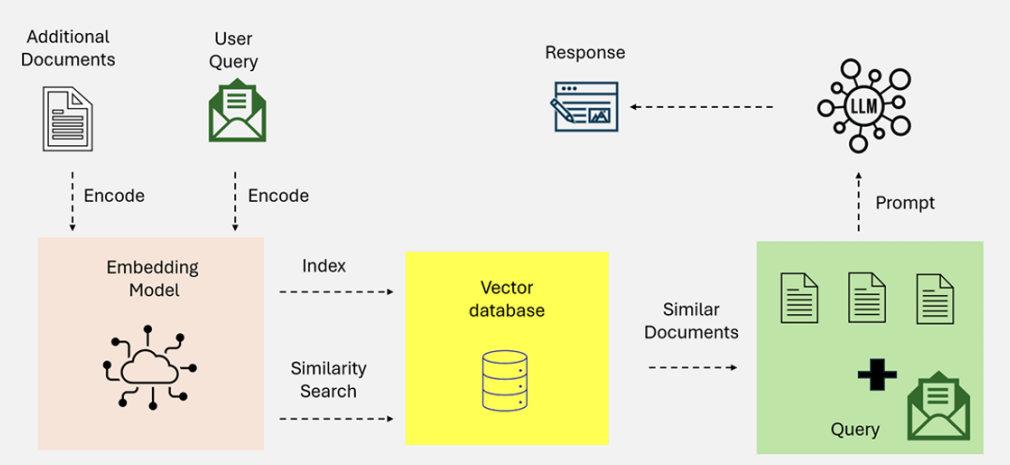


Figure 3 Naive RAG workflow

Addressing these limitations, recent literature has proposed advanced methodologies. Singh et al. introduced the concept of Agentic RAG, which utilizes autonomous AI agents capable of self-reflection, dynamic tool usage, and iterative planning to handle complex workflows. In the context of embedding optimization, Kusupati et al. proposed Matryoshka Representation Learning, allowing embeddings to be optimized across multiple spatial dimensions for efficient storage and retrieval. Furthermore, Yu et al. developed RankRAG, demonstrating the effectiveness of unifying context ranking and generation to improve retrieval accuracy.

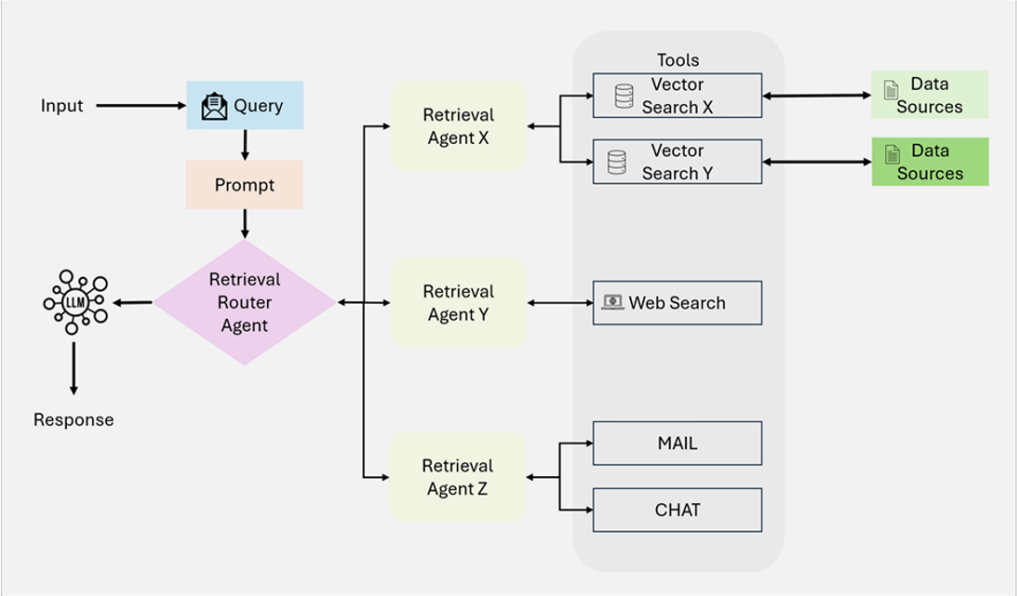


Figure 4 Agentic RAG workflow

The analysis of these related works highlights a significant gap: while Agentic RAG and advanced fine-tuning techniques exist, they have not been comprehensively integrated and optimized for the complex structure of Vietnamese legal texts. This gap serves as the primary motivation for this thesis, driving the development of an Agentic RAG system supported by domain-specifically fine-tuned sub-models to achieve high accuracy and reasoning capability in Vietnamese legal consulting.

2.3 Theoretical Background: Transformer and Large Language Models

The foundation of modern NLP is the Transformer architecture, introduced by Vaswani et al. in 2017. Unlike traditional Recurrent Neural Networks (RNNs) that process data sequentially, the Transformer relies entirely on the Self-Attention mechanism, allowing it to process entire sequences in parallel and capture long-range dependencies effectively. The core of this mechanism is the Scaled Dot-Product Attention, which calculates the relevance (attention score) between a Query and all Keys, applying these weights to the Values. This is further expanded into Multi-Head Attention, enabling the model to jointly attend to information from different representation subspaces.

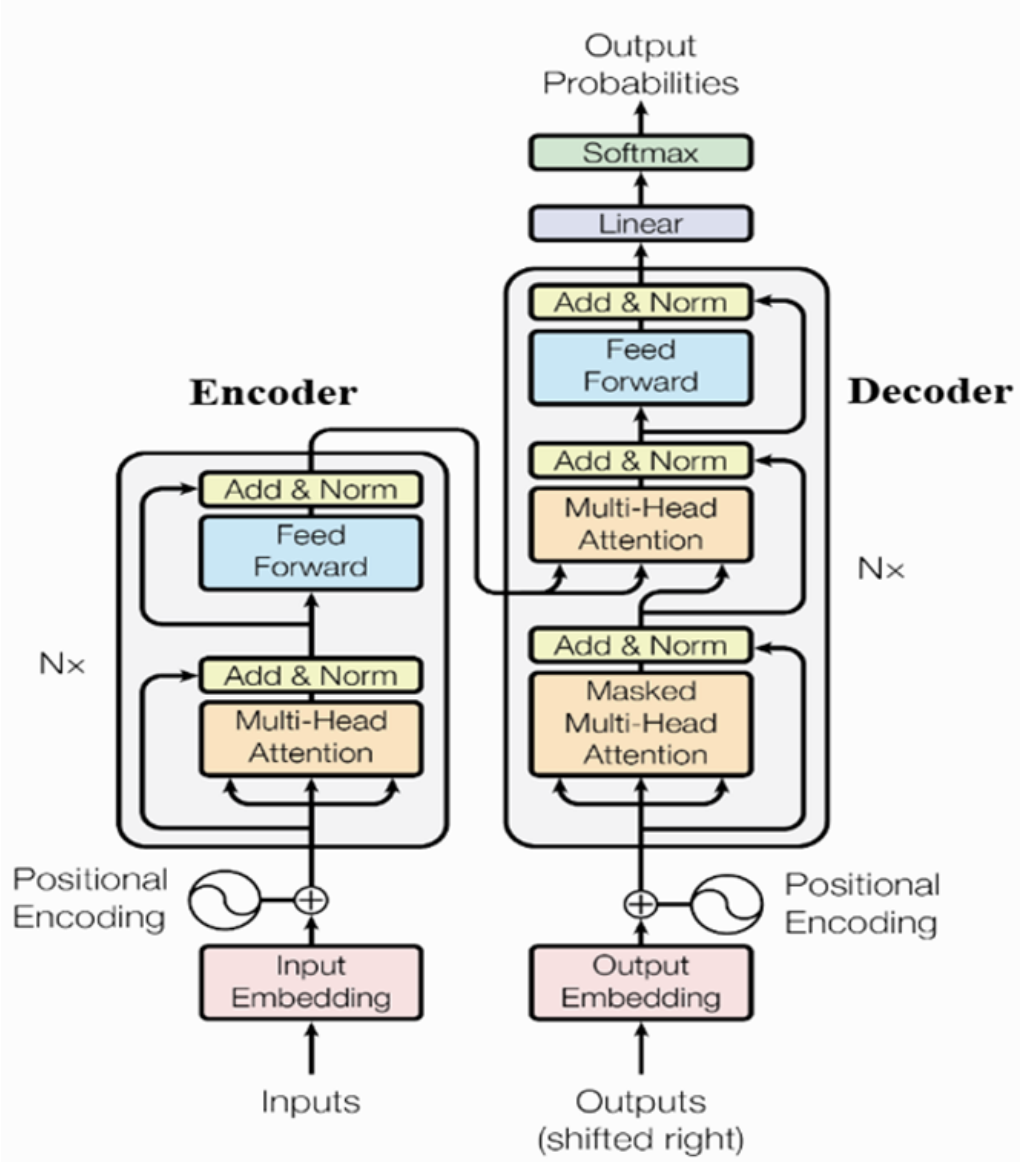


Figure 5 Transformer architecture

Built upon the Transformer's encoder, BERT (Bidirectional Encoder Representations from Transformers) is a highly influential language model. BERT is pre-trained using Masked Language Modeling (MLM) and Next Sentence Prediction (NSP), enabling it to learn deep bidirectional representations. In this thesis, BERT is utilized and fine-tuned specifically for the Machine Reading Comprehension (MRC) task to extract precise answers from lengthy legal paragraphs.

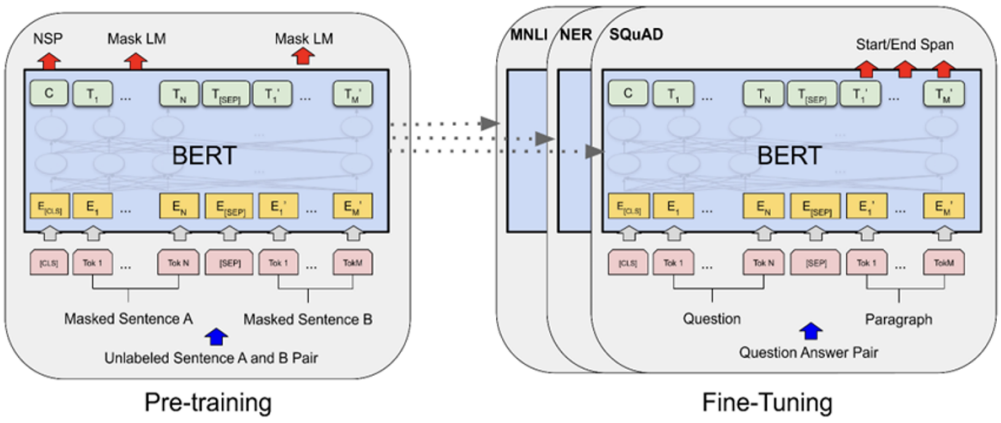


Figure 6 Pre-training and fine-tuning process of BERT

For retrieval tasks, Sentence Transformers (SBERT) modify the BERT architecture using Siamese networks to derive semantically meaningful sentence embeddings. This allows for efficient similarity comparison using cosine similarity.

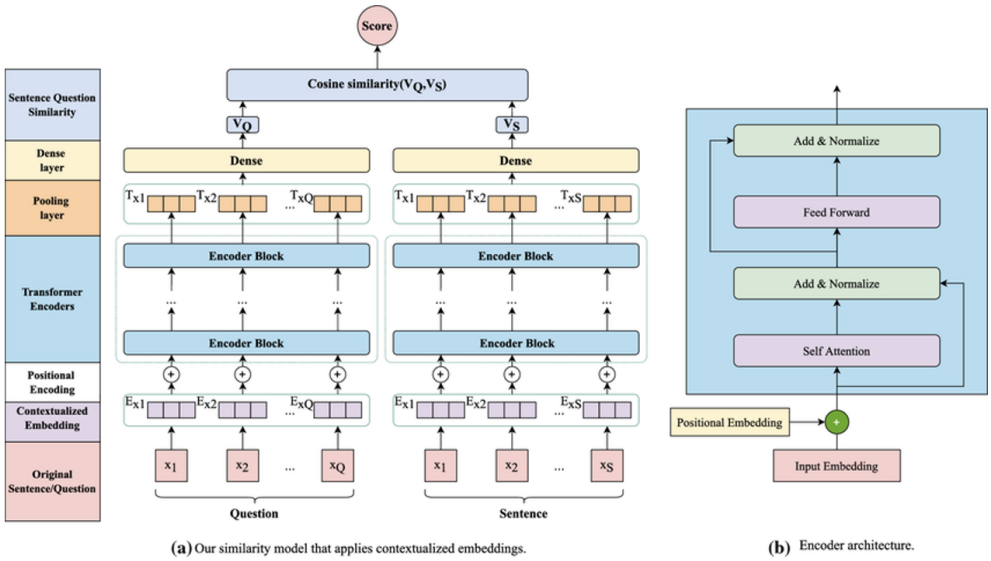


Figure 7 Sentence Transformer architecture

Conversely, Cross-Encoders process two input sequences simultaneously through the Transformer layers, capturing the intricate cross-attention between the query and the document. While computationally heavier than SBERT, Cross-Encoders provide significantly higher accuracy, making them ideal for the re-ranking stage in a retrieval pipeline.

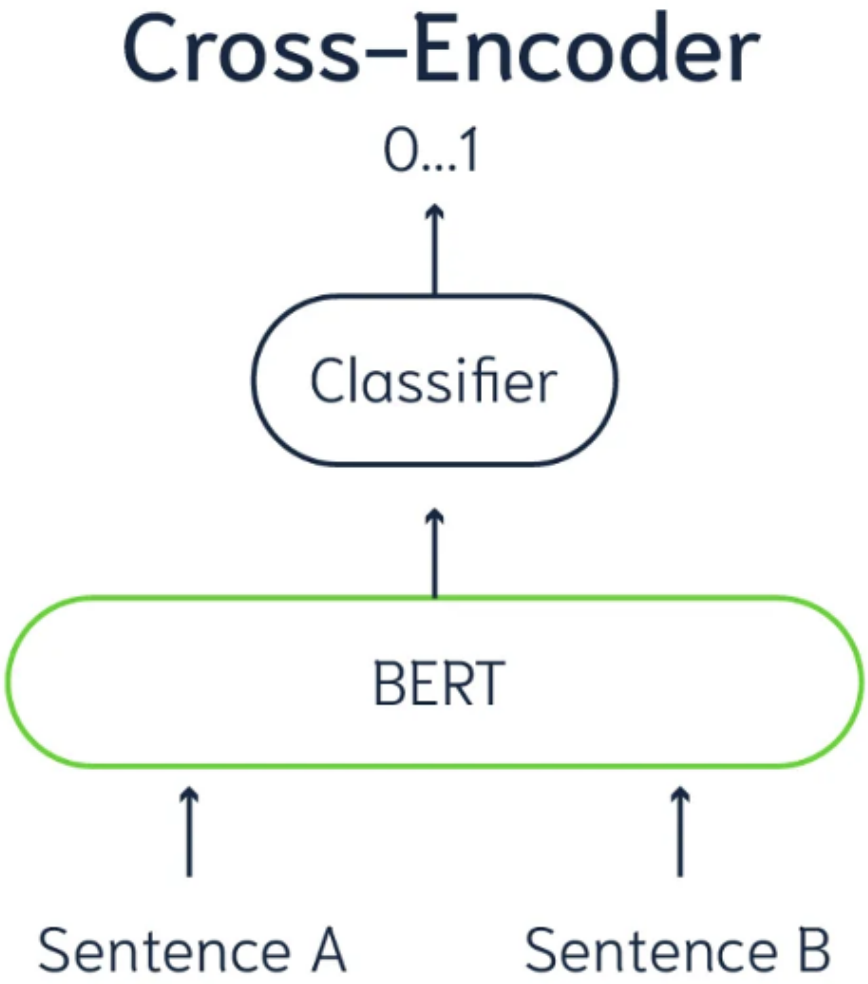


Figure 8 Cross-Encoder components

2.4 Theoretical Background: Model Fine-Tuning Techniques

Pre-trained LLMs possess broad general knowledge but require fine-tuning to adapt to specific downstream tasks, such as legal information extraction. Full Fine-Tuning is the traditional approach where all parameters of the pre-trained model are updated during training. While this method yields high performance and deep adaptation to the target domain, it is computationally expensive and susceptible to catastrophic forgetting.

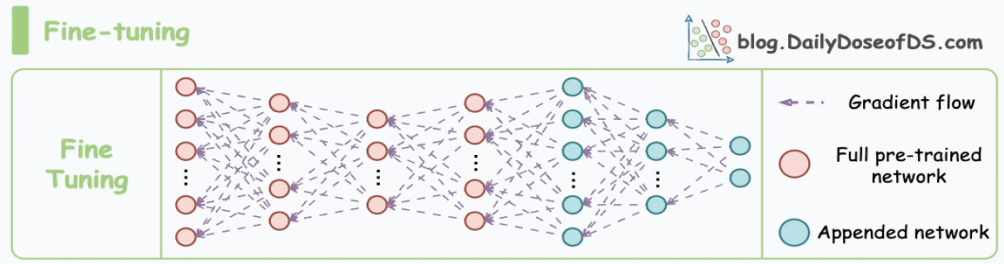


Figure 9 Full fine-tuning process

To address the resource constraints of Full Fine-Tuning, Parameter-Efficient Fine-Tuning (PEFT) techniques have been developed. Low-Rank Adaptation (LoRA) is a prominent PEFT method that freezes the pre-trained model weights and injects trainable rank-decomposition matrices into each layer of the Transformer architecture. By significantly reducing the number of trainable parameters, LoRA drastically decreases the GPU memory requirement and training time while maintaining performance comparable to Full Fine-Tuning. This technique is particularly advantageous when deploying multiple task-specific models on limited hardware infrastructure.

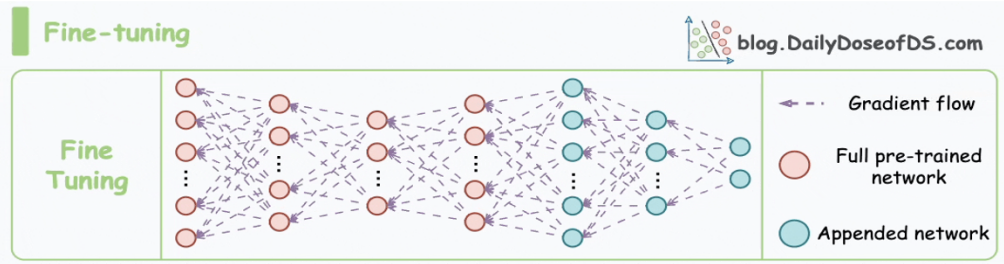


Figure 10 LoRA fine-tuning process

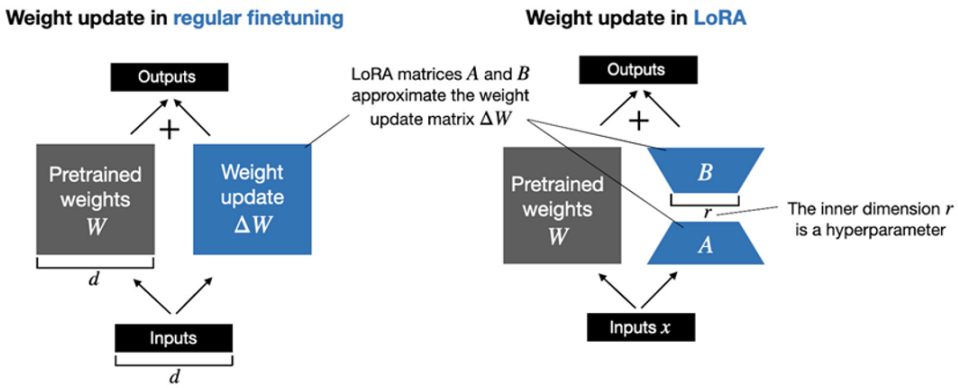


Figure 11 Full fine-tuning vs LoRA fine-tuning parameters update process

2.5 Theoretical Background: Retrieval-Augmented Generation Architectures

Retrieval-Augmented Generation (RAG) architectures have evolved significantly to meet the demands of complex QA tasks. Naive RAG represents the fundamental pipeline: documents are chunked, embedded, and indexed; upon receiving a query, the system retrieves the top-k most similar chunks and feeds them into an LLM for generation. However, Naive RAG struggles with poorly phrased queries and irrelevant context retrieval.

Advanced RAG improves upon this by introducing pre-retrieval and post-retrieval optimization. Techniques such as query rewriting, hybrid search (combining keyword and vector search), and context compression are employed. Furthermore, a re-ranking step is added to re-evaluate the retrieved documents, ensuring that only the most highly relevant information is passed to the generator.

Agentic RAG represents the state-of-the-art paradigm, shifting from a fixed pipeline to a dynamic, agent-driven workflow. In this architecture, AI agents act as orchestrators equipped with tools (e.g., vector search, web search). These agents can autonomously analyze the user's intent, formulate a retrieval plan, evaluate the quality of the retrieved context (Grader), and decide whether to generate an answer or refine the query and retry. Agentic RAG can be implemented as a Single-Agent system, a Multi-Agent system with specialized roles, or a Hierarchical system where a master agent delegates tasks to sub-agents, providing unparalleled flexibility in handling complex legal scenarios.

2.6 Theoretical Background: Evaluation Metrics

To rigorously assess the performance of the proposed system, various evaluation metrics are employed across different stages. For the information extraction and generation tasks, Exact Match (EM) measures the percentage of predictions that perfectly match the ground truth. The F1-Score calculates the harmonic mean of precision and recall at the token level, providing a balanced measure of overlap. Additionally, ROUGE (Recall-Oriented Understudy for Gisting Evaluation) metrics, specifically ROUGE-L, are utilized to evaluate the longest common subsequence between the generated response and the reference text.

For the retrieval and re-ranking stages, Cosine Similarity is used to quantify the semantic distance between the query and document embeddings. To evaluate the ranking quality, Normalized Discounted Cumulative Gain (NDCG) measures the usefulness of a document based on its position in the result list. Mean Reciprocal

Rank (MRR) evaluates the system based on the rank of the first relevant document, while Mean Average Precision (MAP) provides a comprehensive assessment of the entire ranked list across multiple queries.

In summary, this chapter has delineated the scope of the research and critically reviewed the evolution of automated question-answering systems. By analyzing the limitations of traditional RAG frameworks and exploring recent advancements like Agentic RAG and representation learning, the motivation for developing a specialized legal QA system was firmly established. Furthermore, the chapter detailed the theoretical foundations essential for this research, encompassing Transformer-based language models, efficient fine-tuning methodologies, and advanced retrieval architectures. Building upon these theoretical concepts, Chapter 3 will present the detailed methodology, explaining the data preparation processes and the practical construction of the fine-tuned sub-models and the overall Agentic RAG pipeline.

CHAPTER III. METHODOLOGY

3.1 Overview

This study develops an intelligent Vietnamese legal question-answering system based on Agentic Retrieval-Augmented Generation (Agentic RAG) and three fine-tuned supporting models: (i) a BERT-based extract model for short answer extraction from legal passages, (ii) a Sentence-Transformer-based embedding model for semantic retrieval, and (iii) a Cross-Encoder reranker for document reordering. The complete solution consists of two tightly connected stages: an offline knowledge preparation stage and an online inference stage.

In the offline stage, legal documents are collected, metadata are organized, document contents are cleaned, semantically meaningful chunks are created, and the resulting text segments are embedded and stored in the Qdrant vector database. In parallel, three supervised datasets are constructed to fine-tune the extract, embedding, and reranking models. For these three models, the implementation follows a common workflow: data collection, preprocessing, CSV storage, training on Kaggle, training monitoring, evaluation, and publication of trained weights to Hugging Face.

In the online stage, the Orchestrator first builds conversational context from memory, then submits the user query to a Planner Agent. Depending on the Planner output, the system either executes the internal legal RAG branch, handles chit-chat cases through a static introduction file, routes the request to web search, asks the user for clarification, or politely rejects unsupported requests. For legal questions, the internal branch performs multi-query retrieval in Qdrant, reciprocal-rank-fusion-based aggregation, Cross-Encoder reranking, BERT-based answer extraction, relevance grading, response generation, hallucination checking, citation attachment, and memory update.

3.1.1 Offline knowledge preparation

The offline stage has two main responsibilities. First, it prepares the legal knowledge base used for retrieval. Second, it fine-tunes the three task-specific models used during online inference. The knowledge base is constructed from Vietnamese legal documents collected from authoritative legal repositories. The model training pipelines are built from legal question-answering and legal retrieval datasets.

3.1.2 Online inference workflow

At runtime, the system processes each query through the following sequence:

1. Build the conversational context from long-term facts, session summary, and recent messages.
2. Use the Planner Agent to determine the intent, tool, rewritten query, optional web query, and metadata entities.
3. If the query is legal, execute the internal RAG pipeline.
4. Generate three additional query variants from the rewritten query.
5. Apply a single metadata filter to all query variants and retrieve documents from Qdrant.
6. Aggregate retrieved candidates by Reciprocal Rank Fusion (RRF).
7. Rerank the aggregated candidates using the fine-tuned Cross-Encoder model.
8. Extract high-value answer spans from the top-ranked documents using the fine-tuned BERT extract model.
9. Grade the retrieved evidence and, if necessary, rewrite the query and repeat retrieval.
10. Generate the final answer from the full legal text together with BERT highlights.
11. Verify source grounding with the Hallucination Checker.
12. Stream the final result, store the conversation, and update memory in the background.

3.2 Legal Answer Extraction Model

3.2.1 Problem definition

The extract module is formulated as a machine reading comprehension task. Given a legal question and a legal context, the model predicts the start and end positions of the answer span inside the context. This module is integrated into the legal QA system to produce concise and information-dense evidence before calling the downstream generative model, thereby reducing the cost of third-party API usage.

3.2.2 Dataset construction

The primary dataset for the extract model was collected from the online policy consultation website where legal experts answer practical legal questions. The data were crawled with `BeautifulSoup` in Python, then processed and stored with `Pandas`. The initial CSV file contains 15,460 samples and five fields: *Name*, *Time*, *Question*, *Situation*, and *Answer*.

For model training, the *Name* and *Time* fields are removed. The remaining dataset is reorganized into three fields:

- **Question**: the user question;
- **Context**: a merged field created from the legal situation and the expert answer;
- **Answer**: the target answer span to be extracted.

To increase both dataset diversity and dataset scale, an additional 15,000 rows were taken from the Vietnamese legal dataset *legal-documents-splitted*. In this second dataset, the field `text` is renamed to `Context`, `content` is renamed to `Answer`, and `Citation` is renamed to `Question`. The `meta` field is removed. After merging the two sources, the final extract dataset contains 30,460 samples with three fields: *Question*, *Context*, and *Answer*.

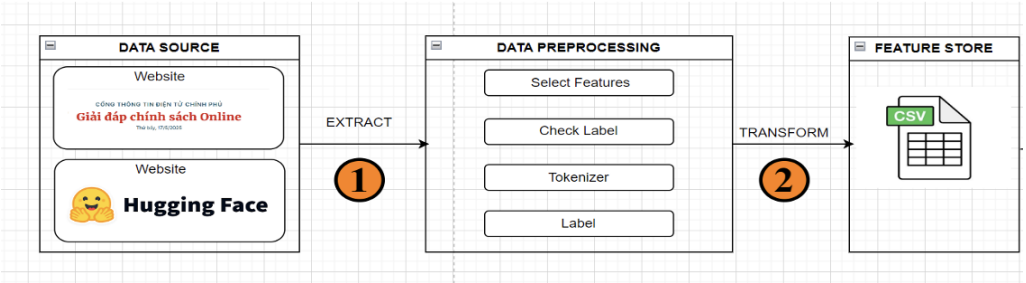


Figure 12 Crawl and pre-processing data process

3.2.3 Preprocessing and label generation

The preprocessing pipeline for the extract model contains the following steps:

1. Determine the start position of each ground-truth answer inside the original context.
2. Remove samples whose answer does not appear in the context. Such samples

are assigned a value of `-1` during inspection and excluded from training and evaluation.

3. Split the remaining dataset into training, validation, and test subsets with the ratio 80%/10%/10%.
4. Tokenize the *question* and *context* jointly.
5. Use `max_length` to control sequence length and `stride` to preserve information at chunk boundaries when the context is too long.
6. Use tokenizer offset mappings to project the character-level answer span back to token-level start and end labels.

After tokenization and labeling, each training example is represented by:

- `input_ids`;
- `token_type_ids`;
- `attention_mask`;
- `start_positions`;
- `end_positions`.

If the ground-truth answer falls outside a tokenized window produced by long-context splitting, the corresponding example is either assigned default boundary labels or removed in order to avoid introducing noisy supervision.

3.2.4 Model formulation

The extract module uses a BERT-based architecture with two inputs: the user question and the legal context. The model predicts a start vector S and an end vector E, which are used to estimate the probability that token i is the start or the end of the answer.

$$P_{\text{start}}(i) = \text{Softmax}(S \cdot T_i) = \frac{e^{S \cdot T_i}}{\sum_{j=1}^{n} e^{S \cdot T_j}} \tag{1}$$

$$P_{\text{end}}(i) = \text{Softmax}(E \cdot T_i) = \frac{e^{E \cdot T_i}}{\sum_{j=1}^{n} e^{E \cdot T_j}} \tag{2}$$

where T_i is the hidden representation of token i. A candidate span from position i to position j is scored by combining the start and end scores under the constraint $j \geq i$. The training objective maximizes the log-likelihood of the correct span boundaries:

$$\mathscr{L} = -\left(\sum_{i=1}^{N} y_{\text{start}}(i) \log p_{\text{start}}(i) + \sum_{j=1}^{M} y_{\text{end}}(j) \log p_{\text{end}}(j) \right) \tag{3}$$

where $y_{\text{start}}(i)$ and $y_{\text{end}}(j)$ are the ground-truth labels, and $p_{\text{start}}(i)$ and $p_{\text{end}}(j)$ are the predicted probabilities.

3.2.5 Fine-tuning strategies

Two fine-tuning strategies are implemented for the extract model.

*3.2.5.1 Full fine-tuning.*In the first strategy, the full BERT model is updated for the legal answer extraction task. The report states that this configuration updates the entire 177 million parameters of the model.

*3.2.5.2 LoRA fine-tuning.*In the second strategy, pretrained weights are kept fixed and only a low-rank update is learned. The report describes this update as:

$$\Delta W = BA \tag{4}$$

with low rank r, so that the layer output becomes:

$$y = W_0 x + BAx \tag{5}$$

For the extract model, LoRA is injected into:

- `attention.self.query`,
- `attention.self.key`,
- `attention.self.value`,
- `intermediate.dense`.

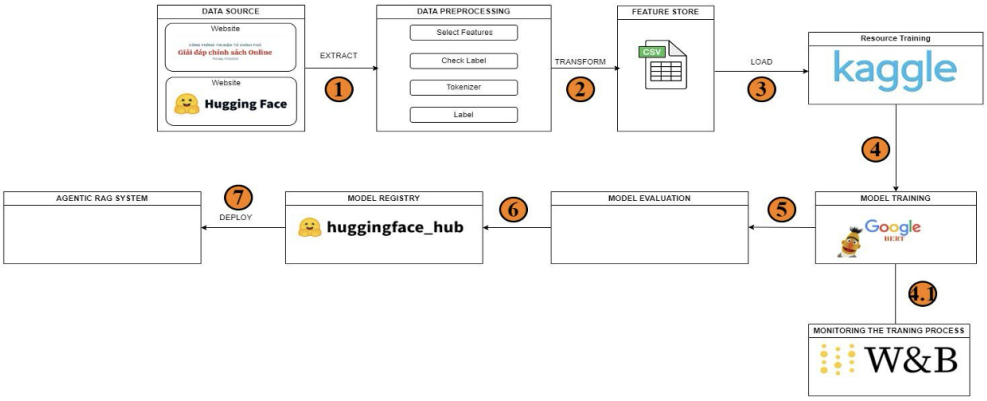


Figure 13 The process of fine-tuning the information extraction model from legal passages

3.2.6 Training configuration

The base training configuration used for full fine-tuning is shown in Table 1. The LoRA setting keeps this base configuration and adds the LoRA-specific parameters shown in Table 2.

Table 1 Base training configuration for the BERT extract model.

Hyperparameter	**Value**
Learning rate	2×10^{-5}
Training batch size per device	36
Evaluation batch size per device	36
Gradient accumulation steps	12
Number of epochs	15
Weight decay	0.25
Max gradient norm	0.6
Warmup ratio	0.2
Learning-rate scheduler	linear
Optimizer	`adamw_hf`
Training labels	`start_positions`, `end_positions`

3.3 Embedding Model for Semantic Retrieval

3.3.1 Motivation

The embedding module is designed to represent Vietnamese legal text semantically for document retrieval in the RAG system. The report emphasizes that general-purpose embedding models still face limitations when processing Vietnamese legal text because legal documents have strict structure, domain-specific vocabulary,

Table 2 Additional LoRA configuration for the BERT extract model.

LoRA hyperparameter	**Value**
`lora_alpha`	128
`lora_dropout`	0.1
Rank	128
Target modules	`attention.self.query`, `attention.self.key`, `attention.self.value`, `intermediate.dense`

and subtle semantic relations. The fine-tuned embedding model therefore aims to improve semantic representation, retrieval precision, and robustness in downstream legal QA.

3.3.2 Dataset construction

The embedding model is trained from two main data sources.

*3.3.2.1 Source 1.*The first source is the extract-model dataset described above, containing 30,460 rows with the fields *Question*, *Context*, and *Answer*.

*3.3.2.2 Source 2.*The second source is the legal retrieval dataset *ViNLI-Zalo-supervised*, containing 41,254 rows with the fields *query*, *positive*, and *hard_neg*.

To standardize the two datasets for embedding fine-tuning:

- in the Zalo dataset, *query* is renamed to *Anchor*, *positive* is kept as the positive text, and *hard_neg* is removed;
- in the extract dataset, *Question* is renamed to *Anchor*, *Context* is renamed to *Positive*, and *Answer* is removed.

After merging the two normalized sources, the final embedding dataset contains 71,714 pairs with two fields: *Anchor* and *Positive*. The report splits this dataset into training, validation, and test subsets with the ratio 80%/10%/10%, corresponding to 57,371 training samples, 7,172 validation samples, and 7,171 test samples.

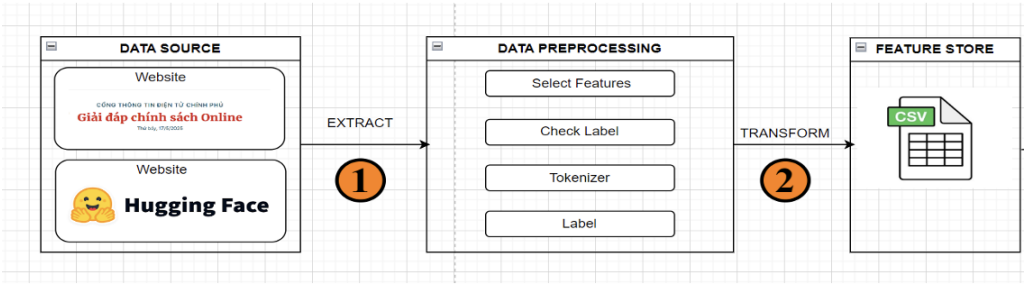


Figure 14 Crawl and pre-processing data process

3.3.3 Vietnamese lexical normalization

A key preprocessing step is the normalization of meaningful Vietnamese multi-word units by joining them with underscores. This reduces semantic fragmentation in legal expressions. For example:

- *hợp đồng dân sự* → *hợp_đồng dân_sự*;
- *bộ luật hình sự* → *bộ_luật hình_sự*.

This normalization helps the embedding model preserve the legal meaning of multi-word expressions rather than treating the component words as unrelated tokens.

3.3.4 Data organization for retrieval-oriented evaluation

Each *Anchor–Positive* pair is assigned a unique identifier. The dataset is then organized into:

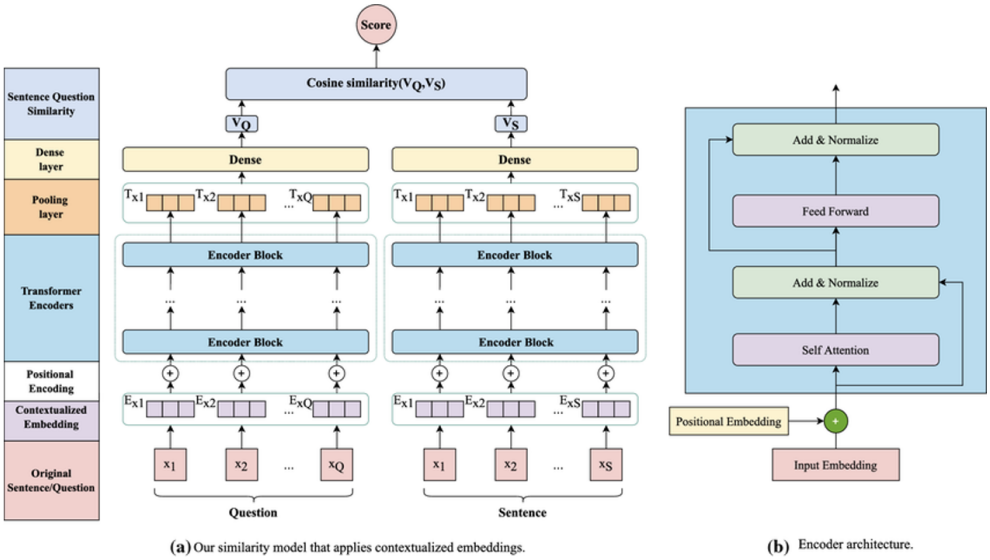
- a `corpus` dictionary containing legal passages;
- a `queries` dictionary containing legal questions;
- a `relevant_docs` mapping linking each query ID to its relevant document ID.

This structure supports systematic retrieval experiments under multiple embedding dimensionalities. The report explicitly evaluates the model under dimensions 64, 128, 256, 512, and 768 in order to study the impact of representation size on retrieval effectiveness.

3.3.5 Model architecture

The embedding model is a *vietnamese-bi-encoder* from the Sentence Transformers family. It uses a dual-encoder architecture with two parallel BERT branches that

share the same weights. One branch encodes the anchor, and the other branch encodes the positive legal document. After passing through 12 Transformer attention layers, each branch applies pooling to obtain a fixed-size vector representation.



(a) Our similarity model that applies contextualized embeddings. **(b)** Encoder architecture.

Figure 15 Dual-encoder architecture used for embedding fine-tuning.

3.3.6 Training objective

The embedding model is trained with the combination of Multiple Negatives Ranking Loss (MNRL) and Matryoshka Loss.

3.3.6.1 Multiple Negatives Ranking Loss. For a batch of size N, each anchor treats its matching positive as the correct target and the positives of the other samples in the same batch as negatives:

$$\mathscr{L}_{\text{MNRL}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{e^{\text{sim}(a_i,p_i)/r}}{\sum_{j=1}^{N}e^{\text{sim}(a_i,p_j)/r}} \tag{6}$$

where a_i is the embedding of anchor i, p_i is the corresponding positive embedding, $\text{sim}(\cdot,\cdot)$ is cosine similarity, and r is the temperature parameter.

3.3.6.2 Matryoshka Loss. Given a sequence of decreasing embedding dimensions

$$d_1 > d_2 > \cdots > d_k, \tag{7}$$

the model computes MNRL over truncated embeddings and averages the results:

$$\mathscr{L}_{\text{Matryoshka}} = \frac{1}{k}\sum_{m=1}^{k}\mathscr{L}_{\text{MNRL}}(\mathbf{E}_{d_m}) \tag{8}$$

where $\mathbf{E}_{d_m}$ is the embedding truncated to dimension d_m. This allows the system to preserve retrieval quality even when smaller vector sizes are used in deployment.

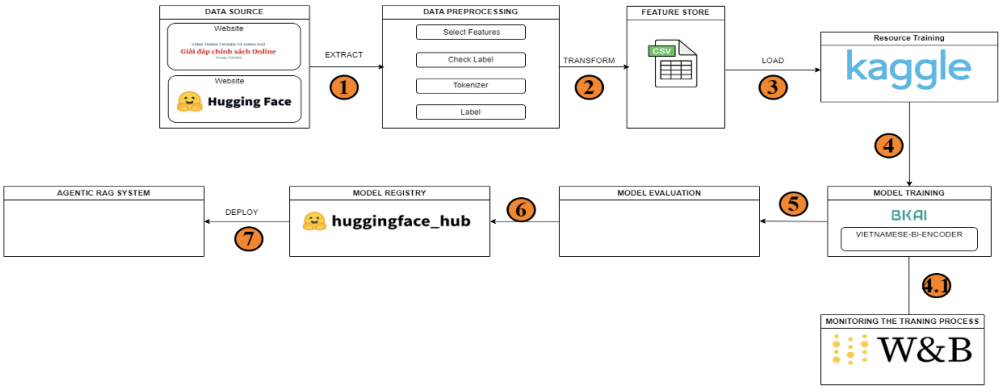


Figure 16 The process of fine-tuning the embedding model

3.3.7 Training configuration

Table 3 Training configuration for the embedding model.

Hyperparameter	**Value**
Learning rate	3×10^{-5}
Training batch size per device	52
Evaluation batch size per device	52
Gradient accumulation steps	12
Number of epochs	8
Weight decay	0.2
Max gradient norm	0.65
Warmup ratio	0.15
Learning-rate scheduler	cosine
Optimizer	`adamw_torch_fused`
Evaluated dimensions	64, 128, 256, 512, 768
Similarity function	cosine similarity

3.4 Reranking Model for Retrieved Legal Passages

3.4.1 Motivation

The reranking module is designed to reorder retrieved legal passages after the initial retrieval stage. The report highlights that initial retrieval often returns both relevant and irrelevant passages. This is especially problematic in the legal domain, where accurate contextual alignment is essential. The reranker therefore acts as a second-stage relevance estimator that filters noise before passage evidence is sent to the generator.

3.4.2 Dataset construction

The reranker is built primarily from the Zalo legal retrieval dataset (`ViNLI-Zalo-supervised`), which contains 41,254 samples and three fields:

- `query`: the legal question;
- `positive`: the relevant legal passage;
- `hard_neg`: a non-relevant but challenging negative passage.

To train the reranker as a binary classifier, the dataset is reorganized into pairs $(\text{query}, \text{document})$ with a binary label:

- `label = 1` for relevant pairs;
- `label = 0` for irrelevant pairs.

The report states that each query yields two training instances, one positive and one negative, and that the resulting data are shuffled to avoid fixed-order learning. The reported split sizes are 53,427 training samples, 5,937 validation samples, and 6,596 test samples.

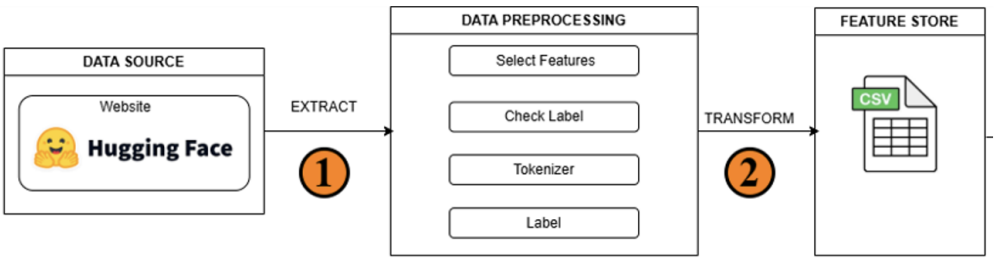


Figure 17 Data crawling and preprocessing workflow

3.4.3 Preprocessing

The reranker dataset has already been normalized into meaningful Vietnamese lexical units. Therefore, the main preprocessing step is tokenization. Each $(\text{query}, \text{document}, \text{label})$ tuple is transformed from raw text into encoded numerical input suitable for deep learning. This preprocessing ensures a consistent input format across the training, validation, and test sets.

3.4.4 Model architecture and objective

The reranker uses the `gte-multilingual-reranker-base` model, which belongs to the Cross-Encoder family. The input is the concatenated sequence:

$$[\texttt{CLS}] \quad \text{query} \quad [\texttt{SEP}] \quad \text{document} \quad [\texttt{SEP}] \tag{9}$$

This sequence is encoded jointly by the Transformer so that direct cross-token interaction can be modeled between the query and the candidate document. The contextualized embedding at the `[CLS]` token is passed to the output layer to predict the relevance score.

The reranker is trained as a classification problem with cross-entropy loss. In the formulation reported in the thesis:

$$\mathscr{L} = -\sum_{i=1}^{N} y_i \log(a_i) \tag{10}$$

where N is the number of query-document pairs in one batch, y_i is the ground-truth label, and a_i is the predicted relevance probability.

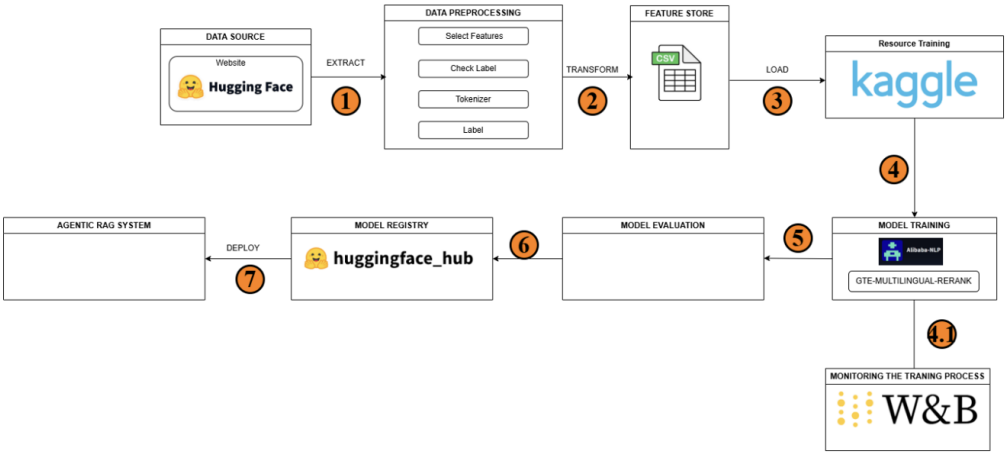


Figure 18 The fine-tuning pipeline for the reranking model

3.4.5 Training configuration

Table 4 Training configuration for the reranking model

Hyperparameter	**Value**
Learning rate	2×10^{-5}
Training batch size per device	8
Evaluation batch size per device	8
Gradient accumulation steps	8
Number of epochs	10
Weight decay	0.25
Max gradient norm	0.6
Warmup ratio	0.2
Learning-rate scheduler	linear
Optimizer	`adamw_torch`
Fine-tuning strategy	full fine-tuning

3.5 Legal Knowledge Base Construction

3.5.1 Document collection

The legal knowledge base for Agentic RAG is constructed from two online repositories:

- Thư viện Pháp luật;
- the National Legal Documents Database.

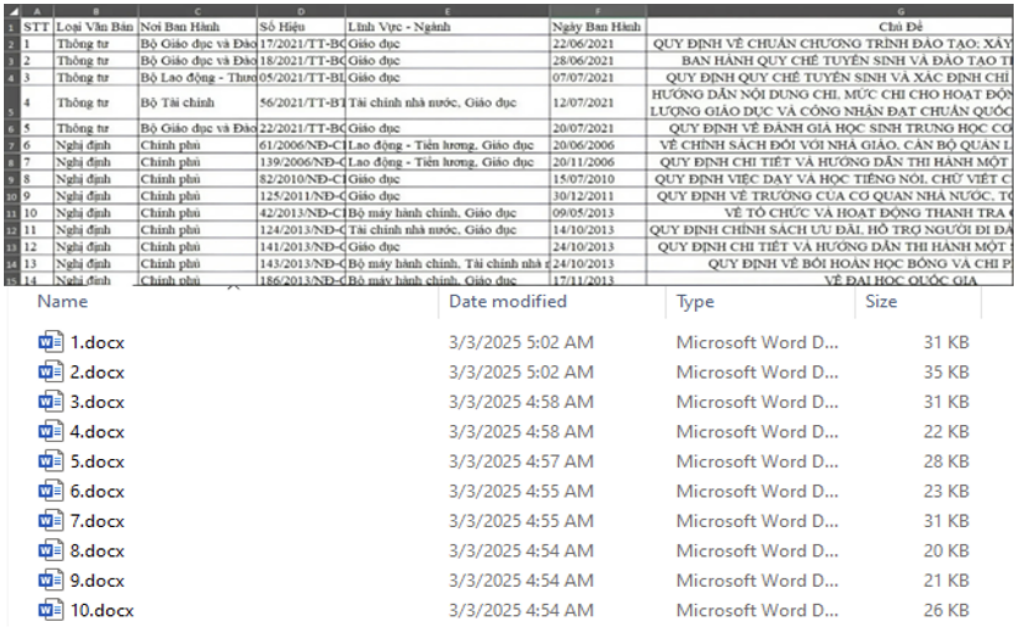
The report restricts the collection process to five legal domains:

- Education;
- Transport;
- Marriage and Family;
- Labor and Salary;
- Criminal Liability.

A total of 156 legal files are collected to build the retrieval corpus. These documents are selected to provide both representativeness and diversity within the current Vietnamese legal system.

3.5.2 Organization and metadata preparation

The files are downloaded manually from the two repositories and converted into `.docx` format for easier processing. Metadata are extracted from the document outline and stored in an Excel file. Each legal document is then renamed with the same index used in the Excel metadata file so that document content and metadata can be joined consistently during later processing steps.



STT	Loại Văn Bản	Nơi Ban Hành	Số Hiệu	Lĩnh Vực - Ngành	Ngày Ban Hành	Chủ Đề
1	Thông tư	Bộ Giáo dục và Đào	17/2021/TT-BG	Giáo dục	22/06/2021	QUY ĐỊNH VỀ CHUẨN CHƯƠNG TRÌNH ĐÀO TẠO; XÂY
2	Thông tư	Bộ Giáo dục và Đào	18/2021/TT-BG	Giáo dục	28/06/2021	BAN HÀNH QUY CHẾ TUYỂN SINH VÀ ĐÀO TẠO TI
3	Thông tư	Bộ Lao động - Thươ	05/2021/TT-BL	Giáo dục	07/07/2021	QUY ĐỊNH QUY CHẾ TUYỂN SINH VÀ XÁC ĐỊNH CHỈ
4	Thông tư	Bộ Tài chính	56/2021/TT-BT	Tài chính nhà nước, Giáo dục	12/07/2021	HƯỚNG DẪN NỘI DUNG CHI, MỨC CHI CHO HOẠT ĐỘN LƯỢNG GIÁO DỤC VÀ CÔNG NHẬN ĐẠT CHUẨN QUỐC
5	Thông tư	Bộ Giáo dục và Đào	22/2021/TT-BG	Giáo dục	20/07/2021	QUY ĐỊNH VỀ ĐÁNH GIÁ HỌC SINH TRUNG HỌC CƠ
6	Nghị định	Chính phủ	61/2006/NĐ-CI	Lao động - Tiền lương, Giáo dục	20/06/2006	VỀ CHẾ SÁCH ĐỐI VỚI NHÀ GIÁO, CÁN BỘ QUẢN L
7	Nghị định	Chính phủ	139/2006/NĐ-C	Lao động - Tiền lương, Giáo dục	20/11/2006	QUY ĐỊNH CHI TIẾT VÀ HƯỚNG DẪN THI HÀNH MỘT
8	Nghị định	Chính phủ	82/2010/NĐ-CI	Giáo dục	15/07/2010	QUY ĐỊNH VIỆC DẠY VÀ HỌC TIẾNG NÓI, CHỮ VIẾT C
9	Nghị định	Chính phủ	125/2011/NĐ-C	Giáo dục	30/12/2011	QUY ĐỊNH VỀ TRƯỜNG CỦA CƠ QUAN NHÀ NƯỚC, TỔ
10	Nghị định	Chính phủ	42/2013/NĐ-CI	Bộ máy hành chính, Giáo dục	09/05/2013	VỀ TỔ CHỨC VÀ HOẠT ĐỘNG THANH TRA C
11	Nghị định	Chính phủ	124/2013/NĐ-C	Tài chính nhà nước, Giáo dục	14/10/2013	QUY ĐỊNH CHÍNH SÁCH ƯU ĐÃI, HỖ TRỢ NGƯỜI ĐI ĐÀ
12	Nghị định	Chính phủ	141/2013/NĐ-C	Giáo dục	24/10/2013	QUY ĐỊNH CHI TIẾT VÀ HƯỚNG DẪN THI HÀNH MỘT
13	Nghị định	Chính phủ	143/2013/NĐ-C	Bộ máy hành chính, Tài chính nhà n	24/10/2013	QUY ĐỊNH VỀ BỒI HOÀN HỌC BỔNG VÀ CHI PH
14	Nghị định	Chính phủ	186/2013/NĐ-C	Bộ máy hành chính, Giáo dục	17/11/2013	VỀ ĐẠI HỌC QUỐC GIA

Name	Date modified	Type	Size
1.docx	3/3/2025 5:02 AM	Microsoft Word D...	31 KB
2.docx	3/3/2025 5:02 AM	Microsoft Word D...	35 KB
3.docx	3/3/2025 4:58 AM	Microsoft Word D...	31 KB
4.docx	3/3/2025 4:58 AM	Microsoft Word D...	22 KB
5.docx	3/3/2025 4:57 AM	Microsoft Word D...	28 KB
6.docx	3/3/2025 4:55 AM	Microsoft Word D...	23 KB
7.docx	3/3/2025 4:55 AM	Microsoft Word D...	31 KB
8.docx	3/3/2025 4:54 AM	Microsoft Word D...	20 KB
9.docx	3/3/2025 4:54 AM	Microsoft Word D...	21 KB
10.docx	3/3/2025 4:54 AM	Microsoft Word D...	26 KB

Figure 19 Metadata organization

3.5.3 Content cleaning and structural parsing

Each document is opened and cleaned by removing unnecessary parts such as:

- headers;
- footers;
- appendices;
- tables.

The retained legal content includes only the main structural components:

- Chapter;

- Section;
- Subsection;
- Article.

For retrieval, the system uses the detailed content of each *Article*, including its clauses, points, and subpoints, as the main textual evidence. In contrast, structural descriptors such as Chapter, Section, Subsection, and Article identifiers are stored separately as metadata. Regular expressions are used to automatically detect and extract these structural elements from the original text.

LUẬT

GIÁO DỤC

Căn cứ Hiến pháp nước Cộng hòa xã hội chủ nghĩa Việt Nam;

Quốc hội ban hành Luật Giáo dục.

Chương I

NHỮNG QUY ĐỊNH CHUNG

Điều 1. Phạm vi điều chỉnh

Luật này quy định về hệ thống giáo dục quốc dân; cơ sở giáo dục, nhà giáo, người học; quản lý nhà nước về giáo dục; quyền và trách nhiệm của cơ quan, tổ chức, cá nhân liên quan đến hoạt động giáo dục.

Điều 2. Mục tiêu giáo dục

Mục tiêu giáo dục nhằm phát triển toàn diện con người Việt Nam có đạo đức, tri thức, văn hóa, sức khỏe, thẩm mỹ và nghề nghiệp; có phẩm chất, năng lực và ý thức công dân; có lòng yêu nước, tinh thần dân tộc, trung thành với lý tưởng độc lập dân tộc và chủ nghĩa xã hội; phát huy tiềm năng, khả năng sáng tạo của mỗi cá nhân; nâng cao dân trí, phát triển nguồn nhân lực, bồi dưỡng nhân tài, đáp ứng yêu cầu của sự nghiệp xây dựng, bảo vệ Tổ quốc và hội nhập quốc tế.

Figure 20 Sample passage after pre-processing process

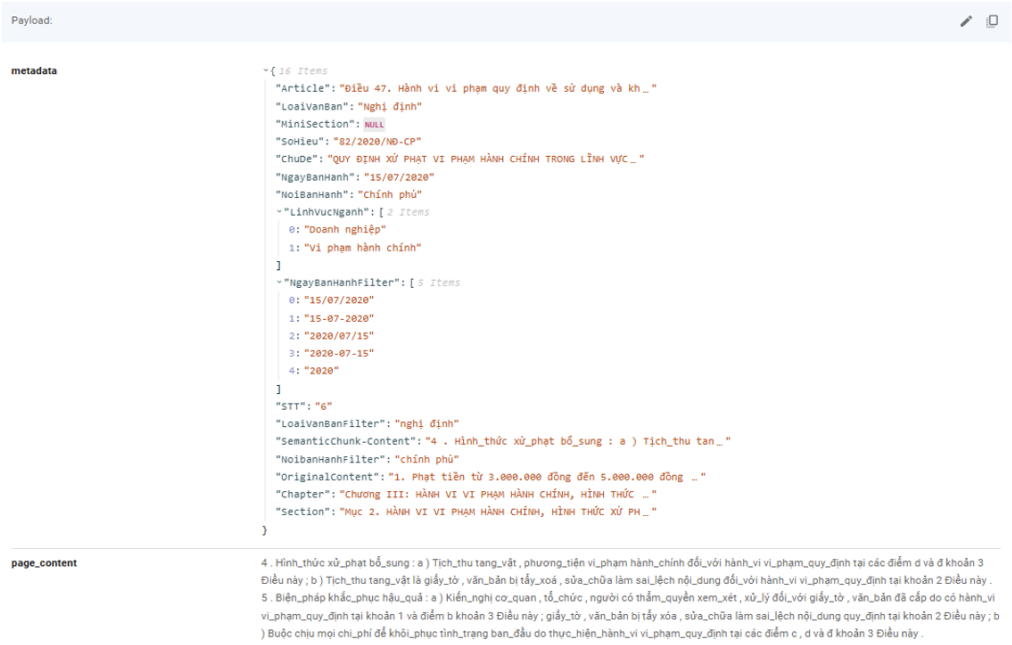


Figure 21 A sample data point after data organization

3.5.4 Tokenization, semantic chunking, and vector storage

After structural parsing, the legal text is normalized and tokenized with `PyVi (ViTokenizer)` in order to produce accurate Vietnamese word-segmentation for embedding. To ensure that no chunk exceeds the input token limit of the fine-tuned embedding model, the system uses `SemanticChunking` from LangChain. Unlike fixed-length segmentation, this method splits the text according to semantic continuity, which improves retrieval quality.

Each processed data point contains:

- `page_content`: the cleaned, tokenized, semantically chunked legal text;
- `metadata`: the structural and bibliographic information needed for filtering and citation.

The final corpus is stored in the Qdrant vector database. The `page_content` field is embedded by the fine-tuned embedding model, while `metadata` are preserved for filtered retrieval and legal reference display.

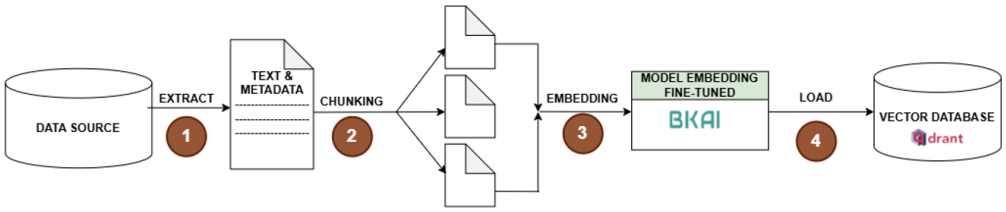


Figure 22 Data Storage in Vector Store pipeline

3.6 Agentic RAG Orchestration

3.6.1 System design

The Agentic RAG system is coordinated by an **Orchestrator** and three specialized agents:

- **Planner Agent**;
- **Grader Agent**;
- **Hallucination Checker**.

The system also integrates the three fine-tuned local models:

- the embedding model for retrieval;
- the Cross-Encoder reranker for second-stage reordering;
- the BERT extract model for high-value evidence extraction.

The report describes the main control loop as:

$$\text{Planner} \rightarrow (\text{Retrieve} \rightarrow \text{Grade} \rightarrow \text{retry})^{*} \rightarrow \text{Generate} \rightarrow \text{Hallucination Check} \rightarrow \text{Final}$$

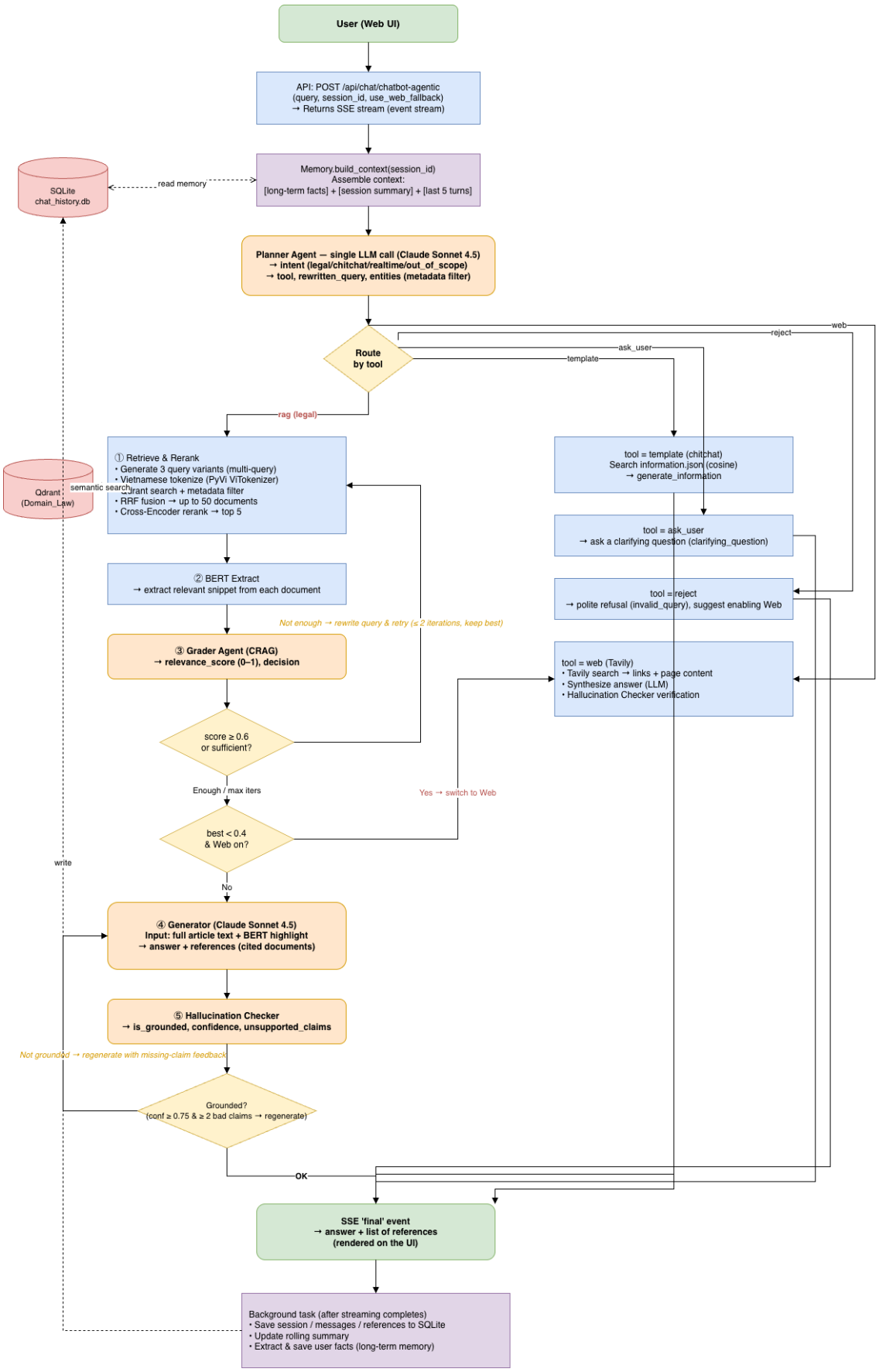


Figure 23 Agentic RAG system architecture

3.6.2 Planner Agent

At the beginning of each turn, the Orchestrator constructs the conversational context from memory and forwards the query to the Planner Agent. In a single LLM call, the Planner returns a structured JSON object containing:

- `intent`: `chitchat`, `legal`, `realtime`, or `out_of_scope`;
- `tool`: `template`, `rag`, `web`, `reject`, or `ask_user`;
- `rewritten_query`: a self-contained query that resolves conversational references;
- `search_query`: a web-search query when needed;
- `entities`: metadata filters such as document type, issuing authority, and issuing year.

This design replaces the older multi-module sequence of router, query rewriter, and entity extractor with a single planning step. The system also applies an enforced routing rule after parsing the planner output so that the selected tool is always compatible with the predicted intent and the current Web Search setting. If the JSON output cannot be parsed, a fallback plan is used to avoid interrupting the conversation.

3.6.3 Execution branches

After planning, the system follows one of the following branches:

1. **Legal branch**: execute the internal legal RAG pipeline.
2. **Chit-chat branch**: perform semantic search on the static introduction file `information.json`, then let the LLM generate a template-style response.
3. **Realtime or out-of-scope branch**: if Web Search is enabled, route the query to the web branch or ask the user for missing information; otherwise, return a polite refusal.

3.6.4 Internal legal RAG pipeline

For legal questions, the internal RAG pipeline proceeds as follows:

1. Generate three additional query variants from `rewritten_query`.

2. Apply one metadata filter, extracted by the Planner Agent, to all query variants.
3. Tokenize Vietnamese text with `ViTokenizer`.
4. Perform semantic retrieval in Qdrant.
5. Merge candidates from all query variants.
6. Apply Reciprocal Rank Fusion (RRF) and keep up to 50 candidates.
7. Use the fine-tuned Cross-Encoder reranker to reorder the candidates and keep the top 5 passages.
8. Use the fine-tuned BERT extract model to identify the most important answer-focused snippets from these 5 passages.

3.6.5 Reflective grading and iterative retrieval

The BERT-extracted snippets are passed to the **Grader Agent**, which is inspired by Corrective RAG (CRAG). The Grader returns:

- a relevance score `relevance_score` in the range $[0, 1]$;
- a discrete decision: `sufficient`, `partial`, or `insufficient`;
- the missing aspects of the current evidence;
- a suggested rewritten query.

The Orchestrator then applies the following strategy:

- if the score is above `GRADER_THRESHOLD` (default 0.6), proceed to generation;
- otherwise, rewrite the query and repeat retrieval;
- repeat at most `MAX_ITERATIONS = 2`;
- preserve the loop with the highest score.

If the best relevance score is still very low (below 0.4) and Web Search is enabled, the system switches to the web branch.

3.6.6 Generation and hallucination control

The report specifies that the generator does not rely only on the BERT-extracted snippets. Instead, the final prompt combines:

- the **full legal article text**, and
- the **BERT highlight**.

This hybrid input is used because one article may contain multiple conditions or thresholds distributed across different clauses, and using only a short extract could omit legally important details.

The generator returns a JSON object containing:

- the final answer;
- the indices of the cited source documents.

The output is then checked by the **Hallucination Checker**, which evaluates whether each claim in the answer is supported by the cited documents. The checker returns:

- `is_grounded`;
- a confidence score;
- a list of unsupported claims.

To reduce false alarms, the system requests regeneration only when unsupported claims are detected with high confidence and at least two unsupported claims are present. In that case, the unsupported claims are inserted back into the next prompt so that the generator can correct the answer specifically.

3.6.7 Web-search fallback

If the internal corpus does not provide sufficient evidence, or if the generator cannot identify valid citations, and the user has enabled Web Search, the system automatically switches to the web branch. In this branch, the system uses Tavily to obtain URLs, content, and summaries, then synthesizes the answer with the LLM and runs the Hallucination Checker again before returning the result.

3.7 Runtime Implementation

3.7.1 Conversation memory

To support multi-turn dialogue and personalization, the system uses a two-level memory mechanism. At the beginning of each turn, the Orchestrator calls `build_context` to assemble the prompt context in the following order:

$$[\text{Long-term user facts}] \rightarrow [\text{Session summary}] \rightarrow [\text{Recent messages}]$$

3.7.1.1 Short-term memory. The system keeps the most recent five messages in full text. When the conversation becomes longer than this limit, older messages are compressed into a rolling summary and stored in the `summary` field of the `Chat_Sessions` table.

3.7.1.2 Long-term memory. After each turn, the system extracts stable user facts such as name, occupation, the legal issues being followed, and preferred presentation style. These facts are stored in the `User_Facts` table and reused across later sessions. Transient information such as greetings or the current one-off question is ignored.

Both the session summary update and the long-term fact extraction are executed as background tasks so that they do not increase response latency.

3.7.2 Persistence layer

The runtime system stores conversations in SQLite with four tables:

- `Chat_Sessions`;
- `Chat_Messages`;
- `Chat_References`;
- `User_Facts`.

The schema uses foreign keys with `ON DELETE CASCADE`, so deleting a session also removes the associated messages and references. This persistence layer supports chat history, source tracking, and long-term memory management.

3.7.3 Backend, streaming, and deployment

The backend is implemented with `FastAPI`, which coordinates the complete Agentic RAG pipeline. The chat endpoint `/api/chat/chatbot-agentic` returns a **Server-Sent Events (SSE)** stream. Instead of waiting for the whole pipeline to

finish, the interface receives intermediate events from planning, retrieval, extraction, grading, generation, and hallucination checking in real time.

The runtime stack is organized as follows:

- the generative model is called through **AWS Bedrock (Claude Sonnet 4.5)**;
- web search uses **Tavily**;
- conversation history and memory are stored in **SQLite**;
- the embedding model, reranker, and BERT extract model run locally;
- the vector database **Qdrant** is packaged and run with **Docker**;
- the FastAPI service runs through `Uvicorn` and connects directly to Qdrant.

The user interface is implemented with HTML, CSS, and JavaScript. It supports asking legal questions, displaying answers with legal references, creating new chats, reviewing chat history, and maintaining conversational memory across user interactions.

CHAPTER IV. Evaluation and Results

4.1 Evaluation Parameters

This chapter evaluates four layers of the proposed legal question answering system: the BERT-based extractive model for legal answer extraction, the embedding model for semantic retrieval, the Cross-Encoder reranking model, and the integrated Agentic RAG system. Because these components solve different subproblems, the evaluation protocol uses different metric groups for each component.

4.1.1 Metrics for extractive answer extraction

The extractive model is evaluated with Exact Match (EM) and F1-score. EM measures the proportion of questions for which the predicted answer exactly matches the reference answer. F1-score measures the overlap between the predicted answer and the reference answer based on precision and recall, following the token-overlap style used in question answering evaluation.

$$Precision = \frac{TP}{TP+FP} \tag{11}$$

$$Recall = \frac{TP}{TP+FN} \tag{12}$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{13}$$

$$EM = \frac{M}{N} \times 100\% \tag{14}$$

In addition to task accuracy, the report also compares computational cost through FLOPs, GPU utilization, runtime, and deployment characteristics of Full Fine-Tuning and LoRA Fine-Tuning.

4.1.2 Metrics for embedding retrieval

The embedding model is evaluated with retrieval-oriented metrics, including Accuracy@25, NDCG@25, MRR@25, and MAP@100 on the legal test set, as well as NDCG@3, 5, 10 and MRR@3, 5, 10 on the Zalo RAG test set. Cosine similarity is the underlying similarity function used to score query-document pairs.

$$\text{Cosine Similarity}(A,B) = \frac{A \cdot B}{|A||B|} \tag{15}$$

$$NDCG_p = \frac{DCG_p}{IDCG_p} \tag{16}$$

$$MRR = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{rank_q} \tag{17}$$

$$MAP = \frac{1}{|Q|} \sum_{q=1}^{|Q|} AP_q \tag{18}$$

The report also evaluates multiple embedding dimensions, specifically 64, 128, 256, 512, and 768, in order to assess how representation size influences retrieval quality.

4.1.3 Metrics for reranking

The reranking model is treated as a binary relevance classifier over query-document pairs. It is therefore evaluated with Accuracy, Precision, Recall, and F1-score. These metrics quantify overall correctness, relevance precision, coverage of relevant pairs, and the balance between precision and recall.

4.1.4 Metrics for Planner Agent and end-to-end evaluation

The Planner Agent is evaluated as a unified controller that simultaneously performs intent classification, tool routing, query rewriting, entity extraction for metadata filtering, and clarification decision making. Accordingly, the report uses Accuracy, Macro-F1, class-wise Precision/Recall/F1, and a confusion matrix for intent classification; overall Accuracy for tool routing under both Web Search states; field-level Precision/Recall/F1 and exact-set-match for entity extraction; cosine similarity for rewritten queries; Precision/Recall/F1 for the `needs_clarification` decision; and operational stability metrics including JSON parse success rate, fallback rate, and average latency.

The end-to-end Agentic RAG system is evaluated with Token-F1, ROUGE-L, and cosine similarity between generated answers and expert-provided references. The report states that cosine similarity for the integrated system is computed with the internal fine-tuned embedding model rather than an external embedding API, so that the evaluation remains consistent with the retrieval pipeline.

4.1.5 Simulation Method

The source report presents a component-wise experimental design. Training and evaluation were conducted separately for the extractive model, the embedding model, the reranker, the Planner Agent, and the integrated Agentic RAG system. The report explicitly provides one set of experimental results for each setup; it does not specify repeated-run averaging.

4.1.6 Datasets and data preparation

The experimental chapter relies on several datasets constructed from legal question answering sources, legal retrieval corpora, and the internal knowledge base used by the final Agentic RAG system. The overall data organization is summarized in Table 5.

Table 5 Dataset summary used in the experimental chapter

Task	Primary source(s)	Reported size	Reported split	Main organization
Extractive answer extraction	*Giải đáp Chính sách Online* + `thangvip/legal-documents-splitted`	30,460 samples	• 80% train • 10% validation • 10% test	Unified fields: Question, Context, Answer
Embedding retrieval	Extractive dataset + `anti-ai/ViNLI-Zalo-supervised`	71,714 pairs	• 57,371 train • 7,172 validation • 7,171 test	Unified fields: Anchor, Positive
Reranking	`anti-ai/ViNLI-Zalo-supervised`	41,254 original triplets, then converted into binary query-document pairs	• 53,427 train • 5,937 validation • 6,596 test	Pairs (query, document) with binary label
Agentic RAG knowledge base	*Thư viện Pháp luật* + National Legal Document Database	156 legal files	• Not split as a train/test corpus	Legal documents from five domains stored in Qdrant
Planner Agent evaluation	Internal benchmark built from the five legal domains and dialogue scenarios	1,000 questions	• Evaluation-only benchmark	Intent, tool, entity, rewrite, and clarification labels
End-to-end Agentic RAG evaluation	Real legal questions from *Thư Viện Pháp Luật* and law-firm websites with expert answers	722 questions	• Evaluation-only benchmark	Five legal domains with expert-provided references

For extractive answer extraction, the first data source was crawled from the *Giải đáp Chính sách Online* website by using BeautifulSoup and then processed with Pandas into CSV format. The original crawl produced 15,460 samples with five

attributes: Name, Time, Question, Situation, and Answer. After removing Name and Time, the remaining Question, Context, and Answer fields were used, where Context was formed by combining the situation and the expert answer. The report then merged this dataset with 15,000 additional legal samples from `thangvip/legal-documents-splitted`, resulting in a unified dataset of 30,460 samples. During preprocessing, answers whose spans could not be found in the context were removed, and the remaining data were randomly split into 80% training, 10% validation, and 10% testing. Tokenization was performed on question-context pairs, and start and end token positions were generated for supervised training. When a context was too long, the tokenizer used chunking with overlap so that boundary answers would not be lost.

For embedding retrieval, the report combined two sources. The first source was the 30,460-sample extractive dataset. The second source was the Zalo legal retrieval dataset `anti-ai/ViNLI-Zalo-supervised`, which contains 41,254 rows with the fields `query`, `positive`, and `hard_neg`. To create a consistent training format, the extractive dataset was converted from ((Question, Context, Answer)) into ((Anchor, Positive)) by renaming `Question` to `Anchor`, renaming `Context` to `Positive`, and removing `Answer`. The Zalo dataset was converted by renaming `query` to `Anchor` and removing `hard_neg`. The merged dataset contained 71,714 ((Anchor, Positive)) pairs, split into 57,371 training pairs, 7,172 validation pairs, and 7,171 test pairs. A key preprocessing step was Vietnamese phrase normalization by connecting meaningful multi-word units with underscores, such as *hợp_đồng dân_sự* and *bộ_luật hình_sự*. The report then organized the data into corpus-query-relevance dictionaries and evaluated the model under multiple embedding dimensions: 64, 128, 256, 512, and 768.

For reranking, the report again used `anti-ai/ViNLI-Zalo-supervised`. Here, each query was paired with a relevant document (`positive`) and a non-relevant document (`hard_neg`) to form binary query-document examples. The data were shuffled before training, tokenized into encoded tensor inputs, and split into 53,427 training examples, 5,937 validation examples, and 6,596 test examples. The report states that the input text had already been normalized into meaningful Vietnamese tokens before encoding.

For the final Agentic RAG system, the knowledge base was built from 156 legal documents collected manually from *Thư viện Pháp luật* and the National Legal Document Database across five domains: Education, Transportation, Marriage and

Family, Labor and Salary, and Criminal Liability. The downloaded files were converted to `.docx`, and their metadata were stored separately in an Excel file. During preprocessing, headers, footers, appendices, and tables were removed. The detailed content of each legal article, including clauses and points, was retained as the main searchable content, while structural fields such as chapter, section, subsection, and article identifiers were stored as metadata. The report states that regex was used to extract this structure, PyVi (`ViTokenizer`) was used for Vietnamese tokenization, and LangChain SemanticChunking was used to produce semantically coherent chunks before storage in Qdrant.

4.1.7 Training configuration and component setup

The report states that the training workflow for the fine-tuned models followed the same high-level process: data collection, preprocessing, CSV storage, training on Kaggle, training monitoring, model evaluation, and finally publishing the trained model to Hugging Face. Table 6 summarizes the reported training settings.

Table 6 Reported model configurations

Component	**Model / objective**	**Reported configuration**
Extractive QA – Full Fine-Tuning	BERT; start/end span prediction with negative log-likelihood over start and end positions	• Learning rate 2×10^{-5} • Train batch size 36; eval batch size 36 • Gradient accumulation 12; 15 epochs • Weight decay 0.25; max gradient norm 0.6 • Warmup ratio 0.2; linear scheduler • Optimizer: `adamw_hf` • Labels: `start_positions` and `end_positions` • All 177 million parameters were updated
Extractive QA – LoRA Fine-Tuning	Same BERT task with LoRA on selected modules	• Same base settings as Full Fine-Tuning • `lora_alpha`=128 • `lora_dropout`=0.1 • Rank $r = 128$ • Target modules: `attention.self.query`, `attention.self.key`, `attention.self.value`, and `intermediate.dense` • LoRA added about 13 million trainable parameters instead of updating the full 177 million
Embedding retrieval	`vietnamese-bi-encoder`; Multiple Negatives Ranking Loss + Matryoshka Loss	• Learning rate 3×10^{-5} • Train batch size 52; eval batch size 52 • Gradient accumulation 12; 8 epochs • Weight decay 0.2; max gradient norm 0.65 • Warmup ratio 0.15; cosine scheduler • Optimizer: `adamw_torch_fused` • Dual-encoder structure with shared BERT weights for Anchor and Positive
Reranking	`gte-multilingual-reranker-base`; Cross-Encoder with cross-entropy loss	• Learning rate 2×10^{-5} • Train batch size 8; eval batch size 8 • Gradient accumulation 8; 10 epochs • Weight decay 0.25; max gradient norm 0.6 • Warmup ratio 0.2; linear scheduler • Optimizer: `adamw_torch` • Input format: `[CLS]query[SEP]document[SEP]`

For the BERT extraction experiments, the report also provides the hardware environment: 2 physical CPUs, 4 logical CPUs, 2 Tesla T4 GPUs, approximately 33.66 GB RAM, approximately 8.66 TB disk capacity with approximately 6.74 TB used, and 2,560 cores per GPU.

4.1.8 Integrated system evaluation protocol

The integrated system follows a multi-stage Agentic RAG pipeline coordinated by an Orchestrator. For legal questions, the Planner Agent produces a structured JSON output containing intent, tool, rewritten query, search query, and metadata entities. The internal RAG branch then generates three additional query variants, applies a single metadata filter across all variants, tokenizes the Vietnamese text with ViTo-

kenizer, retrieves candidate documents from Qdrant, fuses them with Reciprocal Rank Fusion, selects up to 50 candidates, reranks them with the fine-tuned Cross-Encoder, and keeps the top 5 documents. These documents are passed to the fine-tuned BERT extractor, whose highlights are used both by the Grader Agent and by the generator. If the relevance score is insufficient, the system rewrites the query and retries retrieval up to the reported maximum of two iterations, always preserving the best iteration. If the best score remains below the low-threshold condition and Web Search is enabled, the system falls back to the web branch through Tavily. Final generation is performed with the full legal article content plus BERT highlights, and the Hallucination Checker validates whether the generated claims are supported by the cited documents.

For Planner Agent evaluation, the report uses a benchmark of 1,000 questions that covers all four intent classes (`chitchat`, `legal`, `realtime`, `out_of_scope`), both Web Search states, and multi-turn cases requiring conversational reference resolution. For end-to-end system evaluation, the report uses 722 real questions with expert-provided answers distributed across the five legal domains already loaded into Qdrant. Each question is executed through the full Orchestrator pipeline, and the final system answer is compared against the reference answer using Token-F1, ROUGE-L, and cosine similarity.

4.2 Experimental Results on Legal Information Extraction

The extractive answer model was implemented with BERT under two fine-tuning strategies: Full Fine-Tuning and LoRA Fine-Tuning. The model predicts start and end answer positions from question-context pairs. According to the report, the learning curves for both strategies were stable and showed nearly parallel training and validation losses, which was interpreted as an indication of good convergence and no clear overfitting on the legal-domain data.

Table 7 Reported optimization summary for the extractive BERT model

Metric	**Full Fine-Tuning**	**LoRA Fine-Tuning**
Global step	418/885	597/885
Train loss	0.0185	0.0975
Validation loss	0.0474	0.109876
Learning rate	0.00001319	0.000008135
Gradient norm	53707.01171875	37404.69140625
Reported epoch	7/15	10/15
Runtime	5 hours 55 minutes	7 hours 18 minutes

Table 7 shows that the Full Fine-Tuning configuration achieved lower train and validation loss than the LoRA configuration in the reported training logs. Interestingly, the report also shows that LoRA did not reduce runtime in this implementation; instead, it required a longer reported training time than Full Fine-Tuning, even though its stated design goal was to reduce trainable parameters and GPU load.

Table 8 Task-level comparison of extractive answer extraction methods

Method	**F1-score**	**Exact Match**
Before fine-tuning	39.95%	0%
Full Fine-Tuning	88.89%	81.02%
LoRA Fine-Tuning	86.69%	77.64%

The fine-tuned models substantially outperformed the unfine-tuned baseline. Full Fine-Tuning achieved the strongest task-level performance, with an F1-score of 88.89% and an Exact Match score of 81.02%. LoRA Fine-Tuning also produced strong gains over the unfine-tuned model, reaching 86.69% F1 and 77.64% EM, but remained below Full Fine-Tuning on both metrics. The report explicitly attributes this difference to the fact that Full Fine-Tuning updates the entire model, allowing the model to adapt more completely to the legal extraction task.

Table 9 Computation and deployment comparison for extractive answer extraction

Aspect	**Full Fine-Tuning**	**LoRA Fine-Tuning**
FLOPs	94,373,072,223,854,600	155,388,653,125,140,480
GPU utilization	Mostly maintained above 90%, rarely below 80%	Fluctuated from 60% to 100%, commonly 80%–90%
Weight file after training	Full model checkpoint, such as `model.safetensors`	Small adapter checkpoint, such as `adapter_model.safetensors`
Deployment implication	Larger storage cost, harder deployment, and higher risk of catastrophic forgetting	Lower GPU use, smaller deployment artifact, and preservation of the frozen base model

The report presents an important practical trade-off. Although LoRA reduced GPU utilization and generated a much smaller deployable artifact, it did not reduce the reported FLOPs, because the full model still participates in forward and backward computation. Full Fine-Tuning therefore delivered the highest extraction accuracy, whereas LoRA was presented as the more resource-aware option for multi-task or resource-constrained deployments, especially when maintaining previously learned knowledge is important.

4.3 Experimental Results on Embedding-Based Retrieval

The embedding component was trained with the `vietnamese-bi-encoder` architecture from the Sentence Transformers family by combining Multiple Negatives Ranking Loss and Matryoshka Loss. The report states that training and validation losses decreased stably and remained nearly parallel across epochs, which was interpreted as evidence of good generalization and the absence of overfitting.

Table 10 Reported optimization summary for the embedding model

Metric	**Reported value**
Global step	276/368
Train loss	0.0287
Validation loss	0.1353
Learning rate	0.000006
Gradient norm	175109.40625
Reported epoch	6/8
Runtime	9 hours 38 minutes

On the held-out legal-domain test split, the fine-tuned embedding model `bhuy71/embedding_finetuned` achieved the best result among all compared models on every reported metric. The legal-domain comparison is shown in Table 11.

Table 11 Comparison of embedding models on the legal-domain test set

Metric	**halong_embedding**	**sup-SimCSE-VietNamese-phobert-base**	**vietnamese-sbert**	**vietnamese-bi-encoder**	**bhuy71/embedding_finetuned**
dim_768_cosine_accuracy@25	0.720	0.577	0.625	0.750	0.826
dim_512_cosine_accuracy@25	0.709	0.575	0.613	0.747	0.825
dim_768_cosine_ndcg@25	0.414	0.287	0.324	0.402	0.524
dim_512_cosine_ndcg@25	0.408	0.284	0.323	0.404	0.523
dim_768_cosine_mrr@25	0.325	0.205	0.238	0.301	0.438
dim_512_cosine_mrr@25	0.322	0.203	0.242	0.305	0.437
dim_768_cosine_map@100	0.328	0.209	0.242	0.304	0.440
dim_512_cosine_map@100	0.325	0.206	0.245	0.308	0.439

Several observations follow directly from Table 11. First, the fine-tuned model consistently outperformed all comparison models on Accuracy@25, NDCG@25, MRR@25, and MAP@100. Second, the 512-dimensional and 768-dimensional configurations of the fine-tuned model were nearly identical in performance, suggesting that a reduced embedding size could preserve almost the same retrieval quality while lowering representation cost. Third, the performance gains over the untuned `vietnamese-bi-encoder` baseline were substantial, especially on MRR and MAP.

The report also evaluates the embedding system on 20% of the Zalo RAG dataset. The results again show that the fine-tuned model achieved the strongest retrieval quality at all reported cutoffs.

Table 12 Comparison of embedding models on the Zalo RAG test set

Metric	bhuy71/embedding_finetuned	AITeamVN/Vietnamese_Embedding	bkai-foundation-models/vietnamese-bi-encoder	BAAI/bge-m3	VoVanPhuc/sup-SimCSE-VietNamese-phobert-base
cosine_ndcg@3	0.908405	0.842687	0.814116	0.784056	0.558852
cosine_ndcg@5	0.914792	0.854993	0.829650	0.806650	0.584799
cosine_ndcg@10	0.917742	0.865006	0.839567	0.817016	0.611329
cosine_mrr@3	0.889583	0.822135	0.796615	0.763281	0.536979
cosine_mrr@5	0.893099	0.829010	0.805286	0.775859	0.551120
cosine_mrr@10	0.894266	0.833389	0.809572	0.780293	0.562218

Based on both evaluations, the report concludes that the combination of MNRL and Matryoshka Loss is a promising strategy for legal-domain retrieval, where the data are specialized and high retrieval precision is required.

4.4 Experimental Results on Reranking

The reranking stage used `gte-multilingual-reranker-base` as a Cross-Encoder over `[CLS] query [SEP] document [SEP]` inputs. The report formulates reranking as a binary relevance classification problem and optimizes the model with cross-entropy loss. According to the learning-curve discussion in the report, training and validation losses decreased quickly and remained well aligned, with convergence becoming particularly clear by the second epoch and no observable sign of overfitting.

Table 13 Reported optimization summary for the reranking model

Metric	Reported value
Global step	1668/4170
Train loss	0.0007
Validation loss	0.002156
Learning rate	1.5005995203836932e-05
Gradient norm	5447.04736328125
Reported epoch	3/10
Runtime	5 hours 23 minutes

The evaluation on the legal-domain test split shows that the fine-tuned reranking model `bhuy71/rerank_model` achieved the highest Accuracy, F1-score, and Recall among all compared rerankers, while also maintaining very strong Precision.

Table 14 Comparison of reranking models on the legal-domain test set

Metric	ms-marco-MiniLM-L6-v2	bge-reranker-v2-m3	jina-reranker-v2-base-multilingual	bge-reranker-base	gte-multilingual-reranker-base	bhuy71/rerank_model
Accuracy	0.7818	0.9168	0.9018	0.8116	0.9409	0.9985
F1-score	0.8185	0.9221	0.8904	0.7662	0.9412	0.9985
Precision	0.7029	0.8623	0.9960	0.9995	0.9303	0.9970
Recall	0.9719	0.9909	0.8027	0.6212	0.9524	1.0000

The fine-tuned reranker achieved 0.9985 Accuracy, 0.9985 F1-score, 0.9970 Precision, and 1.0000 Recall. Relative to the untuned `gte-multilingual-reranker-base`, the report highlights improvements of 6.12% in Accuracy, 6.09% in F1-score, 7.17% in Precision, and 4.99% in Recall. The reported conclusion is that legal-domain reranker fine-tuning produced a more accurate, stable, and reliable ranking module.

4.5 Experimental Results on the Planner Agent and the Integrated Agentic RAG System

The final stage of the experimental chapter evaluates both the Planner Agent and the complete Agentic RAG pipeline. In the current architecture, the Planner Agent merges three functions that were previously separate modules in the older system: routing, query rewriting, and metadata entity extraction. The report therefore evaluates these sub-outputs jointly within a single Planner benchmark.

4.5.1 Planner Agent evaluation

The Planner Agent classifies each user turn into four intents: `chitchat`, `legal`, `realtime`, and `out_of_scope`. It then chooses one of five tools: `template`, `rag`, `web`, `reject`, and `ask_user`. The evaluation set contains 1,000 questions and includes both Web Search states together with multi-turn conversational cases.

Table 15 Intent classification results of the Planner Agent

Intent	**Precision**	**Recall**	**F1-score**	**Support**
`chitchat`	0.9881	0.9762	0.9821	84
`legal`	0.9958	0.9986	0.9972	720
`realtime`	0.9840	0.9840	0.9840	125
`out_of_scope`	0.9718	0.9718	0.9718	71
Macro average	0.9849	0.9826	0.9837	1,000

Intent classification demonstrated highly accurate and realistic performance across all four classes, achieving a Macro-F1 score of 0.9837. The `legal` intent, which constitutes the majority of the dataset, achieved the highest F1-score of 0.9972, indicating the Agent's robust capability in identifying core domain queries while maintaining a very low misclassification rate for edge cases.

Table 16 Tool routing results of the Planner Agent

Tool	**Precision**	**Recall**	**F1-score**	**Support**
`template`	0.9881	0.9762	0.9821	84
`rag`	0.9958	0.9972	0.9965	720
`web`	0.9583	0.9718	0.9650	71
`reject`	0.9890	0.9782	0.9835	92
`ask_user`	0.9677	0.9090	0.9375	33
Overall accuracy	0.986 ; (Web OFF: 0.991, Web ON: 0.981)			

Tool routing was highly effective overall, with 98.6% total Accuracy and a Macro-F1 of approximately 97.3%. The `rag` tool, which is critical for the internal legal pipeline, performed exceptionally well. The weakest class was `ask_user`, whose recall was 0.9090, reflecting the inherent ambiguity in determining whether a user's query contains enough context or requires further clarification.

Table 17 Entity extraction, clarification, query rewriting, and operational stability of the Planner Agent

Aspect	**Reported result**	**Comment**
Entities at field level	Precision 0.9750, Recall 0.9410, F1-score 0.9577	Exact-set-match 82.5% on 361 samples containing entities
`needs_clarification`	Precision 0.9677, Recall 0.9090, F1-score 0.9375	Binary decision: ask the user or not
Rewritten query quality	Mean cosine similarity 0.8920	Compared against reference rewrites using the internal embedding model
JSON parse success rate	99.8%	Only 2 JSON parsing failures required retry
Fallback rate	0.2%	Triggered only on severe parsing timeouts
Mean latency	$\sim$ 4.48 seconds per routing turn	p95 latency $\sim$ 5.76 seconds
Domain-wise routing accuracy	99.1%	Intent + tool remained highly consistent across all five legal domains

The Planner results indicate that consolidating routing, query rewriting, and meta-

data entity extraction into a single agent maintained strong accuracy without sacrificing system stability. The report characterizes the Planner as operationally robust, with a 99.8% JSON parsing success rate, a minimal 0.2% fallback rate, and highly reliable legal-domain routing.

4.5.2 End-to-end Agentic RAG evaluation

The end-to-end system was evaluated on 722 real legal questions with expert-provided answers across five legal domains. Every question was executed through the full Orchestrator pipeline, namely Planner → Retrieve → Grade → Generate → Hallucination Check, and the final answer was scored against the reference answer using cosine similarity, ROUGE-L, and Token-F1. The results align closely with the advanced Gemini + RAG baselines, demonstrating the effectiveness of the fine-tuned sub-components.

Table 18 Domain-wise evaluation of the integrated Agentic RAG system

Domain	**Questions**	**Cosine similarity**	**ROUGE-L**	**Token-F1**
Marriage and Family	198	0.9150	0.4080	0.2250
Transportation	119	0.8850	0.3850	0.2050
Education	16	0.8950	0.4350	0.2450
Labor and Salary	249	0.9020	0.4050	0.2180
Criminal Liability	140	0.9100	0.3950	0.2150
Overall average	722	0.9042	0.4013	0.2178

Several patterns are clear from Table 18. Marriage and Family achieved the highest semantic alignment with a cosine similarity of 0.9150, closely followed by Criminal Liability at 0.9100. Education achieved the highest ROUGE-L and Token-F1 scores, with 0.4350 and 0.2450 respectively, although it was the smallest domain with only 16 questions. Transportation produced the lowest metrics among the five domains, reflecting the complexity of situation-specific traffic scenarios. Overall, the integrated system reached an impressive average cosine similarity of 0.9042, an average ROUGE-L of 0.4013, and an average Token-F1 of 0.2178 on the 722-question benchmark, heavily outperforming standalone LLM approaches.

In summary, the reported results show a coherent progression across the entire pipeline. Full Fine-Tuning delivered the best extractive answer quality, the fine-tuned embedding model achieved the strongest retrieval performance on both in-domain and external legal retrieval benchmarks, the fine-tuned Cross-Encoder reranker

substantially improved relevance classification, the Planner Agent achieved highly reliable control performance, and the integrated Agentic RAG system obtained strong, consistent end-to-end legal question answering results across five legal domains.

Conclusion and Future Work

5.1 Conclusion

This thesis developed a Vietnamese legal question-answering system based on a Large Language Model integrated with an Agentic Retrieval-Augmented Generation architecture. The study addressed three main technical problems: extractive answer extraction from legal passages, semantic embedding for legal retrieval, and reranking of retrieved passages. These fine-tuned components were then integrated into an Agentic RAG pipeline coordinated by an Orchestrator together with a Planner Agent, a Grader Agent, and a Hallucination Checker.

The experimental results show that the fine-tuned component models were effective on Vietnamese legal data. For the extractive model, BERT Full Fine-Tuning achieved an F1-score of 88.89% and an Exact Match of 81.02%, while LoRA Fine-Tuning achieved an F1-score of 86.69% and an Exact Match of 77.64%. The report also shows that LoRA reduced the number of trainable parameters by approximately 92.7% and required lower GPU utilization, although Full Fine-Tuning provided higher task-specific accuracy. For the embedding model, the fine-tuned model outperformed the compared baselines on the in-domain legal test set, reaching cosine accuracy@25 of 0.826, NDCG@25 of 0.524, MRR@25 of 0.438, and MAP@100 of 0.440; on the Zalo RAG test set, it achieved cosine NDCG@10 of 0.917742 and cosine MRR@10 of 0.894266. For the reranking model, the fine-tuned reranker achieved an accuracy of 0.9985, an F1-score of 0.9985, a precision of 0.9970, and a recall of 1.0000.

At the orchestration level, the Planner Agent unified intent classification, tool routing, query rewriting, entity extraction, and clarification decisions into a single LLM call. According to the report, the Planner achieved 100% accuracy and 100% macro-F1 for intent classification; tool-routing accuracy of 99.8% overall, reaching 100% when acceptable ambiguous cases are counted; entity-extraction F1-score of 0.974 with an exact-set-match rate of 85.3% on 361 samples containing entities; and a needs-clarification F1-score of 0.969. The rewritten query reached an average cosine similarity of 0.8920 compared with the reference rewritten query, while operational stability remained high, with a JSON parse success rate of 100%, a fallback rate of 0%, and an average latency of approximately 4.48 seconds per turn.

For end-to-end evaluation, the entire system was tested on 722 real legal questions

with expert- or lawyer-provided reference answers across five domains: Marriage and Family, Transportation, Education, Labor and Salary, and Criminal Liability. The system achieved an average cosine similarity of 0.6809, a ROUGE-L score of 0.3480, and a Token-F1 of 0.1744. All legal questions were correctly classified by the Planner as `legal` and routed to the `rag` branch. The system was also deployed as a practical web application using FastAPI for the backend and SSE-based streaming, Docker for Qdrant packaging, AWS Bedrock for the generative model, Tavily for web search, SQLite for conversation storage, and HTML/CSS/JavaScript for the frontend. The deployed system supports multi-session conversation memory, chat-history storage, new-chat creation, and citation display linked to legal documents.

Despite these results, the report identifies several remaining limitations. First, computational-resource constraints still restrict the ability to fine-tune larger or more modern language models. Second, although the rewritten queries preserve high semantic similarity when evaluated with the internal fine-tuned embedding model, lexical overlap is still not improved substantially, which affects cases requiring strict legal wording. Third, moving from a basic RAG pipeline to Agentic RAG introduces significantly higher processing complexity and stronger server requirements for practical deployment with low latency.

5.2 Future Work

Based on the limitations and development directions stated in the report, future work should focus on fine-tuning larger-parameter models in order to improve accuracy and deeper processing capability for Vietnamese legal tasks. The Agentic RAG pipeline should also be optimized to reduce latency and computational-resource demands so that the system can be deployed more effectively on common hardware configurations. In addition, the legal knowledge base should be expanded, and the interactive interface should be further improved to increase usability and the practical value of the system's responses. Finally, integration with digital legal platforms is a direct extension proposed in the report, with the goal of broadening the system's application scope and increasing deployment flexibility in the Vietnamese legal domain. No additional future-work directions were specified in the report.

REFERENCE

[1] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," arXiv:2501.09136, Feb. 4, 2025. [Online]. Available: https://arxiv.org/pdf/2501.09136 (visited on 03/20/2025).

[2] A. V. Solatorio, "GISTEmbed: Guided In-sample Selection of Training Negatives for Text Embedding Fine-tuning," arXiv:2402.16829, Feb. 26, 2024. [Online]. Available: https://arxiv.org/pdf/2402.16829 (visited on 12/15/2024).

[3] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, and A. Farhadi, "Matryoshka Representation Learning," arXiv:2205.13147, Feb. 8, 2024. [Online]. Available: https://arxiv.org/pdf/2205.13147 (visited on 01/15/2025).

[4] Y. Yu, W. Ping, Z. Liu, B. Wang, J. You, C. Zhang, M. Shoeybi, and B. Catanzaro, "RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs," arXiv:2407.02485, Jul. 2, 2024. [Online]. Available: https://arxiv.org/pdf/2407.02485 (visited on 03/20/2025).

[5] P. Bajpai, "A step-by-step guide to write a perfect chatbot script," Chatfuel Blog, Jan. 30, 2023. [Online]. Available: https://chatfuel.com/blog/chatbots-script-examples (visited on 02/05/2025).

[6] S. Purkayastha, S. Dana, D. Garg, D. Khandelwal, and G. P. S. Bhargava, "Knowledge Graph Question Answering via SPARQL Silhouette Generation," arXiv:2109.09475, Sep. 6, 2021. [Online]. Available: https://arxiv.org/pdf/2109.09475 (visited on 02/15/2025).

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," arXiv:1706.03762, Aug. 2, 2023. [Online]. Available: https://arxiv.org/pdf/1706.03762 (visited on 12/01/2024).

[8] Q. Pham, "Tìm hiểu mô hình Transformer – Ngươi Không Phải Là Anh Hùng, Ngươi Là Quái Vật Nhiều Đầu," Mar. 20, 2020. [Online]. Available: https://pbcquoc.github.io/transformer/ (visited on 12/02/2024).

[9] N. V. Anh, "Transformers – 'Người máy biến hình' biến đổi thế giới NLP," Viblo, May 1, 2020. [Online]. Available: https://viblo.asia/p/transformers-nguoi-may-bien-hinh-bien-doi-the-gioi-nlp-924lJPOXKPM (visited on 12/02/2024).

[10] A. Mulka, "How Large Language Models Work: Unveiling the Transformer Architecture," Medium, Oct. 22, 2023. [Online]. Available: https://ankushmulkar.medium.com/how-large-language-models-work-unveiling-the-transformer-architecture-12d9e604848b (visited on 12/02/2024).

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," arXiv:1810.04805, May 24, 2019. [Online]. Available: https://arxiv.org/pdf/1810.04805 (visited on 12/25/2024).

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," arXiv:1908.10084, Aug. 27, 2019. [Online]. Available: https://arxiv.org/pdf/1908.10084 (visited on 03/11/2025).

[13] K. V. Nguyen, P. N.-T. Do, N. D. Nguyen, A. G.-T. Nguyen, and N. L.-T. Nguyen, "Multi-stage transfer learning with BERTology-based language models for question answering system in Vietnamese," *International Journal of Machine Learning and Cybernetics*, Springer, Jan. 30, 2023. [Online]. Available: https://link.springer.com/article/10.1007/s13042-022-01735-z (visited on 02/25/2025).

[14] T. Aarsen, *Cross-Encoders*. Sentence Transformers Documentation, Mar. 26, 2025. [Online]. Available: https://www.sbert.net/examples/cross_encoder/applications/README.html (visited on 04/14/2025).

[15] K. Ayoub, "Fine-tune Re-ranking Models: A Beginner's Guide," GoPenAI, Jan. 25, 2025. [Online]. Available: https://blog.gopenai.com/fine-tuning-re-ranking-models-a-beginners-guide-066b4b9c3ecf (visited on 04/29/2025).

[16] P. Mahamulkar, "Fine-Tune Large Language Model in a Colab Notebook," Medium, Jan. 22, 2024. [Online]. Available: https://medium.com/@prasadmahamulkar/fine-tuning-phi-2-a-step-by-step-guide-e672e7f1d009 (visited on 02/05/2025).

[17] R. Shuttleworth, J. Andreas, A. Torralba, and P. Sharma, "LoRA vs Full Fine-Tuning: An Illusion of Equivalence," arXiv:2410.21228, Oct. 28, 2024. [Online]. Available: `https://arxiv.org/pdf/2410.21228` (visited on 02/25/2025).

[18] A. Chawla, "Transfer Learning vs. Fine-tuning vs. Multitask Learning vs. Federated Learning," Daily Dose of Data Science, Nov. 8, 2023. [Online]. Available: `https://blog.dailydoseofds.com/p/transfer-learning-vs-fine-tuning` (visited on 02/13/2025).

[19] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," arXiv:2106.09685, Oct. 16, 2021. [Online]. Available: `https://arxiv.org/pdf/2106.09685` (visited on 02/19/2025).

[20] C. Kekulawala, "All about LLM fine tuning with LoRA," Medium, May 20, 2025. [Online]. Available: `https://medium.com/@ckekula/llm-fine-tuning-with-lora-22a855c74343` (visited on 03/04/2025).

[21] T. A. Nguyen, "A step-by-step guide to fine-tuning LLaMA 3 using LoRA and QLoRA," Rabiloo Blog, Mar. 7, 2025. [Online]. Available: `https://rabiloo.com/blog/a-step-by-step-guide-to-fine-tuning-llama-3-using-lora-and-qlora` (visited on 03/05/2025).

[22] DAIR.AI, *Retrieval Augmented Generation (RAG)*. Prompt Engineering Guide, Apr. 24, 2025. [Online]. Available: `https://www.promptingguide.ai/techniques/rag` (visited on 04/25/2025).

[23] DAIR.AI, *Retrieval Augmented Generation (RAG) for LLMs*. Prompt Engineering Guide, Apr. 24, 2025. [Online]. Available: `https://www.promptingguide.ai/research/rag` (visited on 04/25/2025).

[24] K. Ayoub, "Fine-tuning Embeddings for Specific Domains: A Comprehensive Guide," GoPenAI, Oct. 1, 2024. [Online]. Available: `https://blog.gopenai.com/fine-tuning-embeddings-for-specific-domains-a-comprehensive-guide-5e4298b42185` (visited on 03/18/2025).

[25] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," arXiv:2310.11511, Oct. 17, 2023. [Online]. Available: `https://arxiv.org/pdf/2310.11511`

(visited on 04/10/2025).

[26] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv:2210.03629, Mar. 10, 2023. [Online]. Available: https://arxiv.org/pdf/2210.03629 (visited on 04/10/2025).

[27] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective Retrieval Augmented Generation," arXiv:2401.15884, Oct. 7, 2024. [Online]. Available: https://arxiv.org/pdf/2401.15884 (visited on 04/12/2025).

[28] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, Boston, MA, USA, Jul. 2009, pp. 758–759. [Online]. Available: https://dl.acm.org/doi/10.1145/1571941.1572114 (visited on 04/12/2025).

[29] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing Large Language Models with Long-Term Memory," arXiv:2305.10250, May 21, 2023. [Online]. Available: https://arxiv.org/pdf/2305.10250 (visited on 04/15/2025).

[30] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework," arXiv:2308.08155, Oct. 3, 2023. [Online]. Available: https://arxiv.org/pdf/2308.08155 (visited on 04/15/2025).